

Worst-Case Discovery and Runtime Protection for RL-Based Network Controllers

Hongyu Hè
Princeton University

Minhao Jin
Princeton University

Maria Apostolaki
Princeton University

Abstract

RL-based controllers achieve strong average-case performance in networking tasks such as congestion control and adaptive bitrate streaming. Yet their performance can degrade severely under network conditions where strong performance is still achievable. Identifying such conditions and quantifying the resulting performance gap is intractable by enumeration, while the sequential and closed-loop nature of RL controllers makes formal verification methods impractical.

We present REGUARD, a framework that discovers worst-case scenarios for a given RL controller and protects it against them at inference time without retraining. REGUARD formulates scenario discovery as a bilevel regret-maximization problem, certifying a lower bound on the controller's worst-case performance gap. REGUARD then analyzes the discovered counterfactual trajectories and compiles them into lightweight logic rules that intervene only when a risky state is detected, leaving the controller's behavior unchanged otherwise.

We evaluate REGUARD across three RL-based network controllers: Pensieve, Sage, and Park. REGUARD discovers scenarios in which each controller suffers a 43–64% performance loss relative to what is achievable under the same conditions. REGUARD exposes performance gaps 57% to 6× larger than those found by the strongest baselines and shrinks those gaps by 79–85% in its first iteration via lightweight rule-based protection, iteratively driving the remaining gap toward nominal levels. REGUARD's protection extends beyond the scenarios it discovers, improving performance across a wide range of network conditions.

1 Introduction

For more than a decade, RL-based and self-adapting controllers have been proposed for networked and cloud systems, including adaptive bitrate streaming, congestion control, and resource management [2, 22, 27, 38, 56, 57]. On standard benchmarks, these learned controllers consistently outperform handcrafted heuristics, making a strong case for production deployment. Yet before an operator can justify deploying a learned controller in production, she needs answers to three questions that average-case benchmarks cannot provide. First, are there realistic network scenarios, meaning sequences of network conditions, under which the RL controller performs far worse than what is achievable? Second, when such scenarios arise, how much achievable performance does the controller forfeit? Third, can we mitigate these worst-case failures at runtime without sacrificing the controller's strong performance under normal operating conditions?

Answering these three questions is difficult because the controller and the network interact in a closed loop over time. An RL controller is a black-box neural policy whose current action alters future network states, such as queue occupancy and packet loss, which in turn shape the observations the controller receives at subsequent steps. Because the space of multi-step network scenarios grows exponentially with the control horizon, existing tools struggle to either find true worst-case failures or fix them safely. Formal verifiers and symbolic analyzers [14, 24, 44, 48] require an explicit mathematical model of both the policy and the packet-level environment dynamics, which is rarely available for production systems, while scripted testing tools such as packetdrill [11] check only fixed packet traces. Approximating an RL controller with a decision tree or surrogate model [28, 41] loses fidelity in the tail regimes where worst-case failures hide. Conversely, black-box fuzzers such as CC-Fuzz [49], AdvNet [4], and Gilad et al. [16] search for conditions that minimize raw controller reward, which often surfaces pathological network outages where no policy can succeed [6]. On the mitigation side, curriculum-learning and fine-tuning frameworks such as Genet [56] and Mowgli [3] update the neural policy's weights globally, which frequently degrades performance on normal traffic. Meanwhile, point solutions such as Canopy [58] and Galileo [51] rely on domain-specific input-output constraints for congestion control or queueing formulas for RPC scheduling that do not generalize across networking tasks.

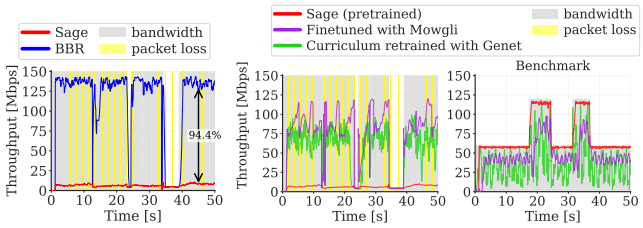
We present REGUARD, a general framework that discovers avoidable worst-case failures and protects black-box RL con-

trollers at runtime without requiring a formal system model or policy retraining [18, 23, 25, 26]. Our first key insight is that a genuine controller failure is defined not by low absolute reward, but by high *regret*, meaning a large gap between what the RL controller achieves and the best performance achievable under the exact same network conditions. Measuring regret separates avoidable controller mistakes from harsh network conditions where every algorithm fails. However, computing the exact optimal policy for every candidate scenario is intractable in closed-loop environments. To make discovery tractable, REGUARD formulates scenario search as a bilevel regret-maximization problem over feasible network conditions. The outer loop uses an RL-based solver to search for the scenario that maximizes the target controller's regret, while the inner loop approximates the scenario-specific optimum using the best-performing policy from a portfolio of established networking heuristics. Because decades of networking research have produced heuristics that excel in complementary operating regimes, their per-scenario upper envelope closely approximates the true optimum and yields a certified lower bound on the controller's worst-case regret.

Our second key insight is that the same heuristic portfolio that certifies a worst-case failure also provides the counterfactual supervision needed to fix it at runtime. Whenever REGUARD discovers a high-regret scenario, it compares the failing RL controller's trajectory against the trajectory of the heuristic that succeeded under those same conditions. This comparison reveals which observable controller states are risky and whether the controller acted too aggressively or too conservatively at each step. REGUARD distills these state-level counterfactuals into compact, threshold-based logic rules [23, 25] that evaluate only the controller's current observation vector at inference time. When no rule matches, REGUARD abstains and leaves the RL controller's action untouched, preserving its nominal performance; when a risky state is detected, REGUARD applies a bounded correction in the direction validated by the counterfactual heuristic.

We evaluate REGUARD across three state-of-the-art RL controllers spanning distinct networking domains: Pensieve for adaptive bitrate streaming, Sage for congestion control, and Park for load balancing. REGUARD discovers feasible scenarios in which these controllers suffer a 43–64% performance loss relative to the reference portfolio, exposing gaps up to 7× larger than those found by Genet [56], Indago [9], and Gilad et al. [16]. At runtime, REGUARD's learned rules shrink these worst-case gaps by 79–85% in a single iteration while preserving or slightly improving performance on normal traces, and two additional refinement rounds drive the remaining gap close to nominal levels.

Critically, REGUARD's protection generalizes beyond the scenarios used to learn the rules, confirming that it repairs underlying policy flaws rather than memorizing specific traces. Controllers protected by REGUARD outperform their unprotected counterparts on challenging scenarios generated by Genet, Indago, and Gilad et al., even though those scenarios were never seen during rule synthesis. Re-running REGUARD's bilevel search against the protected controller likewise yields much smaller worst-case regret, showing that REGUARD shrinks the controller's overall vulnerability surface. Finally, because REGUARD evaluates its rules through a compact prefix trie over raw state features, it fits well within each controller's real-time decision budget and produces human-readable rules that expose concrete policy bugs, such as Pensieve's tendency to select unsustainable bitrates during bandwidth drops and Park's bias toward overloaded slow servers.



(a) Vanilla Sage (b) Fine-tuned Sage

Figure 1: Sage suffers severe, avoidable throughput collapse under specific network conditions, and fine-tuning fails to fix the gap without hurting normal performance. (a) Under a feasible lossy scenario, vanilla Sage [59] achieves 94% lower throughput than BBR. (b) Fine-tuning Sage on challenging scenarios neither restores achievable throughput in that scenario nor preserves Sage's original throughput on normal benchmarks.

2 Motivation

This section uses RL-based congestion control as an example use case to explain why strong benchmark performance is insufficient for real-world RL deployment in networked systems and to derive the requirements for REGUARD. We detail the use cases considered in Table 1 and Appendix B.

2.1 Use Case: RL-Based Congestion Control

Consider an operator managing a private datacenter network. To improve performance for latency-sensitive and high-throughput workloads such as distributed machine-learning training [21, 61], she considers replacing the currently deployed BBR controller with Sage [59], a state-of-the-art RL-based congestion controller that achieves higher average throughput and lower latency. She evaluates Sage on representative internal traces, and the average results are encouraging. However, strong average performance does not rule out severe tail failures in production. As Fig. 1a illustrates, under a feasible scenario where high bottleneck bandwidth is paired with intermittent packet loss, BBR sustains near-capacity throughput whereas Sage overreacts to the loss events, backs off

sharply, and forfeits 94% of the achievable throughput. Before deploying Sage, the operator therefore needs to answer a fundamental question: **Under what feasible network scenarios does Sage fail, and how much achievable performance can it lose?**

Her first instinct is formal verification, which can certify network properties when an explicit mathematical model of the system is available [14, 24, 44, 48]. For instance, QuASI answers queue-related queries from coarse packet counts [48], MetaOpt and whiRL verify policies within simplified analytical models [14, 44], and TypoNet translates static network configurations into symbolic logic [24]. However, verifying Sage's closed-loop throughput in emulation or production would require formally modeling its multi-layer neural network alongside Linux kernel pacing, ACK clocking, queue evolution, and time-varying link conditions over hundreds of RTTs, which is intractable in practice.

Her next thought is adversarial machine learning, which stress-tests neural networks using gradient-based input perturbations such as projected gradient descent (PGD) [10, 17, 35, 37]. However, PGD perturbs Sage's 69-dimensional observation vector directly without respecting the physical laws that couple those features (Table 2). For example, PGD can simultaneously increase Sage's observed delivery-rate ratio and its packet-loss ratio to values that imply full link utilization alongside massive queue drops on an uncongested link, a state that no real bottleneck queue can ever produce. While static classifiers and offline trace generators can enforce snapshot feature constraints or data invariants [8, 19, 25, 26, 30, 31, 47], an RL controller operates in a closed loop where each action changes the future network state and subsequent observations. Ensuring that an adversarial perturbation corresponds to a physically realizable trajectory requires executing or modeling the entire closed-loop environment, bringing the operator back to the same barrier that blocked formal verification.

Her remaining option for discovery is random or heuristic fuzzing. Because a scenario unfolds over T control intervals with multiple link parameters per step, the space of feasible trajectories grows exponentially with T. Random sampling spends nearly all of its budget on benign scenarios where every policy succeeds or on degenerate outages where every policy fails, offering no guarantee of finding selective blind spots like Fig. 1a and no bound on how close the worst observed gap is to the true worst case.

Even if the operator discovers a severe failure scenario like Fig. 1a, she still needs a way to fix Sage without breaking its normal behavior. The standard remedy in machine learning is fine-tuning or curriculum retraining on the failure scenarios [33, 34, 36, 53, 54]. Because neural network weights are shared across all inputs, however, gradient updates that push Sage to send more aggressively during lossy scenarios globally shift its policy across benign regimes as well. Fig. 1b demonstrates this trade-off: fine-tuning Sage on challenging scenarios only partially recovers throughput in the failure scenario of Fig. 1a while degrading its throughput on normal benchmarks where the original model excelled. The operator is left with an unacceptable dilemma: **Should she optimize the controller for normal-case performance or for worst-case robustness?**

2.2 Requirements & Design Principles

The congestion-control example highlights the need for a framework that complements pretrained RL controllers to improve their robustness and deployability.

First, the framework must discover feasible network scenarios under which the target controller performs substantially worse than what is achievable under the same conditions. This gap captures the true worst case for the controller, isolating settings where the controller fails to adapt even though strong performance remains possible. For instance, the scenario in Fig. 1a, where Sage achieves 94% lower throughput than BBR, represents an avoidable failure that an operator must identify and mitigate before deployment.

Second, the protection mechanism must preserve the controller's strong nominal performance. Any mitigation that degrades normal-case performance defeats the purpose of deploying an RL controller, as illustrated by the fine-tuning degradation in Fig. 1b.

Finally, the framework must remain practical to deploy without requiring a formal model of the controller, the environment, or their closed-loop dynamics in advance.

3 Overview

Following these design principles, REGUARD takes the form shown in Fig. 2. Next, we highlight the main insights that drove its design.

By framing discovery as regret maximization over a bilevel program, we can provide a tightness guarantee on how close the discovered failure is to the true worst case. To find scenarios that leave the most achievable performance on the table, we guide the search toward worst-case regret, defined as the performance gap between what the controller achieves and what is achievable under the same conditions. REGUARD frames discovery as an optimization over all feasible network conditions for the scenario that maximizes regret. Because REGUARD optimizes regret directly rather than sampling randomly, the returned scenario carries a formal bound (Theorem 1): the gap between the regret it achieves and the true worst-case regret is bounded by the suboptimality of the reference policy. Computing the exact optimal policy is intractable for a stochastic dynamical system over long horizons, yet decades of networking research have produced heuristics that each excel in different operating regimes. The best-performing heuristic on each scenario forms an upper envelope that serves as a strong proxy for the scenario-specific optimum. REGUARD therefore uses a bilevel structure in

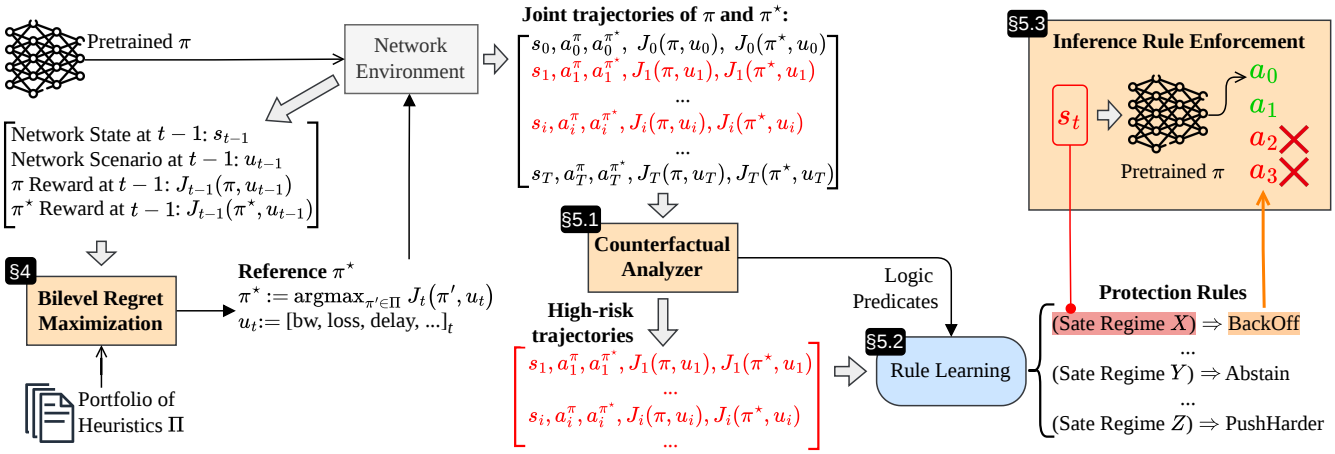


Figure 2: REGUARD discovers scenarios that maximize the targeted RL controller's regret by directly interacting with the network environment in which the controller was trained and by using a portfolio of heuristics to approximate the scenario-specific optimum. It then analyzes the resulting counterfactuals to extract recurring risky patterns and corrective directions. Finally, REGUARD compiles those counterfactuals into logic rules that override the controller's decision at inference time only when intervention is strictly necessary.

which the inner problem selects the best reference heuristic for each candidate scenario that the outer problem explores.

The same upper envelope that certifies the failure also provides the counterfactual signal needed for protection. The portfolio of heuristics serves a second role beyond certification. When REGUARD discovers a high-regret scenario, it identifies both which heuristic succeeded and what that heuristic did differently at each decision point. This comparison provides a concrete counterfactual: under the same network conditions, the reference heuristic backed off while the RL controller pushed harder, or vice versa. By comparing the controller and reference trajectories, REGUARD pinpoints risky states and determines the corrective direction needed to prevent performance degradation. To cover distinct blind spots in the controller, REGUARD searches for a diverse set of high-regret scenarios rather than relying on a single trajectory.

The highest-regret scenarios cluster around a few recurring and high-risk decision-making flaws, making surgical intervention both possible and effective. The highest-regret scenarios reflect systematic flaws in the controller's policy rather than isolated anomalies. Because the search objective rewards the same controller weakness whenever it surfaces, many discovered scenarios expose the same underlying mistake. For instance, REGUARD scenarios reveal that Park repeatedly routes small jobs to slow servers even when fast servers sit idle. Because the discovered scenarios cluster around a small number of recurring state patterns, REGUARD can distill them into a compact set of logic rules over interpretable predicates that cover a wide range of failure conditions while remaining auditable by operators.

4 Bilevel Regret Maximization over Stochastic Dynamical Systems

4.1 Problem Formulation

Evaluating whether an RL controller exhibits a severe performance failure requires an informative frame of reference: if an environment is so degraded that no algorithm succeeds, poor performance reflects an unavoidable barrier rather than a controller flaw. A genuine failure occurs only when the network allows high performance, yet the learned controller forfeits it. REGUARD formalizes this principle by searching for a feasible scenario in $\mathcal{E}$ that maximizes the regret of a fixed pretrained controller π relative to the best reference policy in class Π under that same scenario.

Scenario, reward, and regret. A scenario is a finite resource sequence $e = (u_0, \ldots, u_{T-1}) \in \mathcal{E}$, where each $u_t \in \mathcal{U}$ specifies the exogenous network resources or workload inputs available at step t. Thus, e represents a time-varying network trajectory rather than a static snapshot. For any control policy π', its expected cumulative reward in scenario e, the scenario-specific optimal reference policy $\pi_e^\star$ within reference class Π, and the resulting regret $R(e)$ of the pretrained controller π are defined as

$$\begin{aligned} J(\pi'; e) &:= \mathbb{E}\left[\sum_{t=0}^{T-1} \gamma^t r_t \,\middle|\, \pi', e\right], \\ \pi_e^\star &\in \arg\max_{\pi' \in \Pi} J(\pi'; e), \\ R(e) &:= J(\pi_e^\star; e) - J(\pi; e). \end{aligned} \tag{1}$$

Measuring regret $R(e)$ isolates genuine controller failures. If a scenario is intrinsically bad for every policy, then both $J(\pi_e^\star; e)$ and $J(\pi; e)$ are low and the regret stays small. A large

regret instead means that the controller leaves substantial reward on the table in a feasible scenario where much better performance is still achievable.

Feasible search space. Operators define $\mathcal{E}$ to capture realistic deployment constraints and exclude unrepresentative conditions:

$$\mathcal{E} := \{e : e \models c_j,\ j = 1, \ldots, m\}. \tag{2}$$

Constraints c_j encode operational bounds (*e.g.,* bounding packet loss $0 \leq \text{loss}_t \leq 0.01\%$, capping maximum link bandwidth [21], limiting rates of capacity change, or restricting job arrival envelopes [55]), concentrating discovery on realistic operating regimes rather than pathological artifacts.

Bilevel regret maximization. REGUARD targets the following bilevel problem:

$$e^\star \in \arg\max_{e \in \mathcal{E}} \left[J(\pi_e^\star; e) - J(\pi; e) \right], \tag{3}$$

or equivalently,

$$\begin{aligned} \underset{e \in \mathcal{E}}{\text{maximize}} \quad & J(\pi_e^\star; e) - J(\pi; e) \\ \text{subject to} \quad & \pi_e^\star \in \arg\max_{\pi' \in \Pi} J(\pi'; e). \end{aligned} \tag{4}$$

The outer level searches over feasible scenarios, while the inner level picks the strongest reference policy for each candidate scenario. A high-scoring scenario is therefore a concrete failure case in which the controller underperforms by a large and avoidable amount.

This formulation operates over full trajectories rather than static snapshots. The outer variable e is a time series of resources, and both $J(\pi; e)$ and $J(\pi_e^\star; e)$ depend on the closed-loop evolution over the horizon, where abrupt bandwidth shifts and delayed congestion feedback govern how controllers adapt [42, 61]. Unlike static-input analyses such as MetaOpt [44], REGUARD searches over time-varying network scenarios for dynamical controllers.

Outer RL solver and practical reference. We use RL only as an offline solver for the outer search over scenarios. This outer solver is distinct from the control policies π and $\pi_e^\star$, which act *within* a fixed scenario. RL fits the outer search naturally because the scenario itself is sequential and the search requires no explicit formal model of the pretrained controller or the network dynamics.

In practice, the exact inner solution may be unattainable, especially in continuous-action settings. We therefore use an approximate reference policy $\hat{\pi}_e^\star \in \Pi$ and solve

$$\hat{e} \in \arg\max_{e \in \mathcal{E}} \left[J(\hat{\pi}_e^\star; e) - J(\pi; e) \right]. \tag{5}$$

The theoretical target is Eqn. (4), while the implemented procedure approximately solves Eqn. (5).

4.2 Guarantees

The bilevel formulation provides a formal guarantee with a clear operational meaning: the scenario returned by REGUARD certifies a lower bound on the pretrained controller's worst-case avoidable performance gap over the feasible space $\mathcal{E}$.

For readability, define

$$R(e) := J(\pi_e^\star; e) - J(\pi; e), \qquad \hat{R}(e) := J(\hat{\pi}_e^\star; e) - J(\pi; e). \tag{6}$$

Here $R(e)$ is the exact regret objective and $\hat{R}(e)$ is its practical approximation.

Main guarantee. Under the assumptions stated in Appendix A, let $\tilde{e}$ be the scenario returned by the outer RL solver, let $e^\star$ maximize the exact objective in Eqn. (3), let ε denote the outer-solver suboptimality for Eqn. (5), and let $\delta(e)$ bound the inner-reference error in scenario e. Then

$$R(\tilde{e}) \geq \max_{e \in \mathcal{E}} R(e) - \varepsilon - \delta(e^\star), \tag{7}$$

or equivalently, $\max_{e \in \mathcal{E}} R(e) - R(\tilde{e}) \leq \varepsilon + \delta(e^\star)$.

Equation (7) shows that the exact regret achieved by the returned scenario lower-bounds the worst-case exact regret over the feasible search space. When the outer RL solver is near-optimal and the practical reference is accurate, this lower bound is tight, making the returned scenario a near-tight certificate of controller weakness in the searched regime.

Supporting results. The proof relies on two lemmas proved in Appendix A. First, the outer RL solver returns an ε-optimal scenario for the approximate objective:

$$\hat{R}(\tilde{e}) \geq \max_{e \in \mathcal{E}} \hat{R}(e) - \varepsilon. \tag{8}$$

Second, the approximate regret is a pointwise lower bound on the exact regret, with gap controlled by the inner-reference error:

$$0 \leq R(e) - \hat{R}(e) \leq \delta(e), \qquad \forall e \in \mathcal{E}. \tag{9}$$

Combining Eqns. (8) and (9) yields Theorem 1 below.

> **Theorem 1.** *Under A1–A4, the scenario $\tilde{e}$ returned by* REGUARD *certifies a lower bound on the worst-case exact regret objective, and this lower bound is within $\varepsilon + \delta(e^\star)$ of the exact worst case (Eqn. 7).*[1]

Scope and interpretation of the guarantee. Theorem 1 certifies that REGUARD discovers at least the observed avoidable regret, approaching the true worst case as the outer solver and reference approximation improve. The bound loosens only through two separable offline errors: the outer search error ε, which reflects solver convergence, and the inner reference error $\delta(e^\star)$, which reflects portfolio coverage. Expanding Π reduces $\delta(e^\star)$ by capturing complementary heuristic strengths across regimes [46], tightening the certificate at the cost of

[1] Proof in Appendix A.6.

offline evaluation compute. Crucially, this computation runs entirely offline; neither Π nor the outer solver executes at inference time.

4.3 Implementation

High-level design. Table 1 summarizes how REGUARD instantiates Eqn. (5) across three use cases detailed in Appendix B: congestion control, adaptive bitrate streaming, and load balancing. Each implementation exposes a small set of scenario variables to the outer RL solver, keeps the pre-trained controller π fixed, and constructs a practical reference policy $\hat{\pi}_e^\star$ under the same generated scenario. At time t, the solver observes the controller-induced system state, chooses the next resource element u_t, and appends it to the scenario $e = (u_0, \ldots, u_{T-1})$. REGUARD then evaluates π and the reference under this shared scenario and uses the resulting regret from Eqn. (5) as the learning signal.

To ensure a fair and diagnostic comparison across policies, REGUARD evaluates every policy under the exact same exogenous scenario while keeping each policy's closed-loop state isolated. Using the application-specific reward in Table 1, the outer solver is rewarded only for finding conditions where π performs poorly while $\hat{\pi}_e^\star$ still achieves high reward. Each implementation defines $\hat{\pi}_e^\star$ as the best policy in its practical reference class:

$$J(\hat{\pi}_e^\star; e) := \underset{\pi' \in \Pi}{\text{maximize}}\ J(\pi'; e). \tag{10}$$

Eqn. (10) defines the reference used by the implementation over its chosen Π, approximating the scenario-specific optimum in Eqn. (4). For each application, Π is a portfolio of complementary heuristics whose upper envelope serves as a strong practical reference [3, 46, 59], avoiding dependence on any single algorithm's blind spots. As summarized in Table 1, the portfolios pair delay- and loss-based controllers for Sage (BBR, Cubic, NewReno), buffer- and rate-based policies for Pensieve (BBA, RobustMPC, rolling oracle), and load- and duration-aware heuristics for Park (LLF, SJF, LCT). A stronger $\hat{\pi}_e^\star$ sharpens the reward signal for the outer solver and reduces $\delta(e)$, tightening the certificate in Theorem 1.

Congestion control. Instantiating REGUARD for Sage [59] (Fig. 3, with Pensieve and Park detailed in Appendix C.1) requires process-level emulation because REGUARD evaluates a real Linux congestion-control stack rather than a standalone simulator. REGUARD extends Mahimahi [45] from a passive trace player into an online-controlled, multi-policy emulator. At each outer step, u_t specifies the bottleneck link conditions for the next control interval. The common training mode maps a normalized solver action logarithmically to a shared bottleneck bandwidth, while richer configurations also control loss and propagation delay. This logarithmic bandwidth mapping gives the solver fine resolution in both low-capacity and high-capacity regimes because transport behavior scales with bandwidth ratios rather than absolute Mbps increments.

Sage uses a best-of-portfolio reference over conventional congestion controllers. REGUARD runs Sage, BBR, Cubic, and NewReno under the same link evolution, then applies Eqn. (10) using the normalized transport reward in Table 1. The reward combines delivery rate relative to available capacity, RTT inflation relative to the base RTT, and packet loss normalized by capacity, while leaving multi-flow fairness to future work. Normalizing these terms makes scores comparable across the full bandwidth range that the solver generates. The objective in Eqn. (5) then favors selective performance failures, such as bandwidth transitions, latency regimes, or loss bursts where Sage underreacts, overreacts, or recovers sluggishly while at least one conventional controller remains effective.

The central challenge in Sage is synchronized multi-policy emulation. REGUARD launches one isolated Mahimahi child instance per policy, with separate ports, actor identifiers, runtime directories, TCP state, queue dynamics, and shared-memory observation channels. Isolation prevents one controller's packets, queues, or control history from contaminating another controller's trajectory. At the same time, REGUARD coordinates these child instances as a single logical experiment by applying the same u_t to every child at the same logical step and effective timestamp. This lockstep execution ensures that every policy experiences the exact same link change before the performance gap is scored.

The Mahimahi extension uses an online control plane that updates per-direction bandwidth, delay, and loss dynamically while returning detailed queue and departure telemetry. REGUARD monitors this telemetry to detect stale settings or desynchronized updates and aborts inconsistent steps rather than scoring misleading gaps. Inside each solver interval, REGUARD also supports deterministic loss over short sub-intervals, allowing the solver to express bursty or temporally correlated loss patterns while keeping u_t compact.

5 Protecting RL Controllers against Performance Failures

REGUARD uses the scenarios found by the search in §4.3 to learn lightweight inference-time protection for the pre-trained controller. Rather than retraining the controller or running reference heuristics online, REGUARD distills controller-reference comparisons from challenging scenarios into interpretable rules that decide when to trust the controller and when to apply a bounded correction. This design turns failure discovery directly into runtime protection: the search exposes where the controller leaves reward on the table, and REGUARD converts those failures into targeted interventions that preserve nominal behavior elsewhere.

Use Case	RL Type	Training Environment	Network Scenario Variables	Reward (performance): $J(\pi; e)$	Reference Policy
CCA (Sage [59])	Offline	Emulation	Bandwidth, loss, delay	$\sum_t (\alpha \cdot \text{rate}_t + \beta \cdot \text{rtt}_t - \gamma \cdot \text{loss}_t)$	Best performance among Cubic, BBR, and NewReno.
ABR (Pensieve [38])	Online	Simulation	Bandwidth	$\sum_t (\alpha \cdot b_t - \beta \cdot r_t - \gamma \cdot \lvert b_t - b_{t-1} \rvert)$	Exact K-step rolling bitrate oracle.
LB (Park [39])	Online	Simulation	Job arrival time, job size	$-\alpha \cdot \int_0^T N_{\text{active}}(t)\, dt$	Best performance among LCT, JSQ, and CFS.

Table 1: Implementation summary for the instantiations of Eqn. (5) for the three use cases described in Appendix B. For Sage, $\alpha = 0.60$, $\beta = 0.25$, and $\gamma = 0.15$; rate_t is delivery rate divided by path capacity and clipped to $[0, 1]$, rtt_t is base RTT divided by current RTT and clipped to $[0, 1]$, and loss_t is loss normalized by path capacity and clipped to $[0, 1]$. For Pensieve, b_t is bitrate in Kbps, r_t is rebuffering time, $\alpha = \gamma = 10^{-3}$, and $\beta = 4.3$. For Park, reward is negative active-job-time cost, so maximizing reward minimizes accumulated active-job time. $N_{\text{active}}(t)$ is the number of active jobs, T is the episode horizon, and $\alpha = 1/\texttt{reward_time_scale}$. LCT denotes Least Completion Time, JSQ denotes Join Shortest Queue, and CFS denotes Choose Fastest Server.

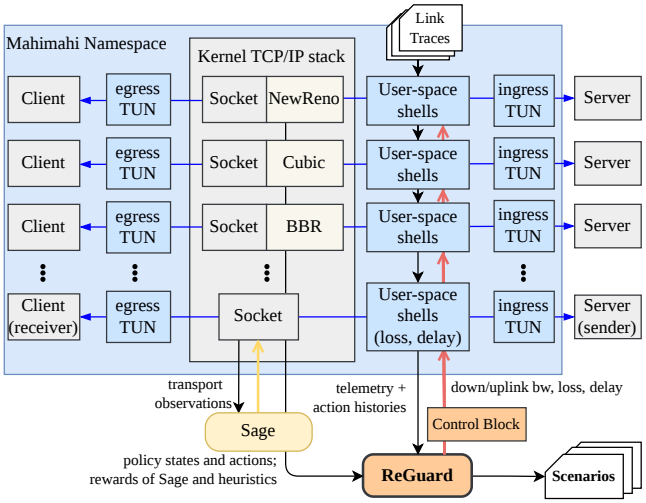


Figure 3: The key design point of Sage's instantiation is end-to-end counterfactual consistency: REGUARD runs Sage and the reference controllers in synchronized but isolated emulation, then reuses the resulting controller-reference trajectories to identify risky states and derive small corrections.

5.1 Identifying Risky Control States

Failure discovery alone does not reveal which individual decisions caused the performance gap. A high-regret scenario shows that the controller underperforms across an episode, but it does not specify which states are risky, whether the controller acted too aggressively or too conservatively, or how the action should change. REGUARD bridges this gap with a *Counterfactual Analyzer* (Fig. 2), which treats the reference-policy portfolio as a set of counterfactual teachers. For each decision point t, the analyzer records the controller state s_t, the controller action $a_t = \pi(s_t)$, the reference action induced by $\hat{\pi}_e^\star$, and the rewards achieved under the same scenario.

The Counterfactual Analyzer converts trajectory-level failures into state-level supervision. It identifies states where the controller is consistently suboptimal, compares the controller action with the reference action under the same observed state, labels the corrective direction, and summarizes recurring discrepancies as operator-meaningful patterns. Specifically, a visited state s_t in scenario e is marked risky only when the episode regret exceeds a threshold τ_{regret} and the reference policy outperforms the controller from that state:

$$\text{Risky}(s_t) = 1 \iff J(\hat{\pi}_e^\star; e) - J(\pi; e) \geq \tau_{\text{regret}}. \tag{11}$$

This criterion keeps REGUARD's protection focused on avoidable controller mistakes rather than intrinsically harsh conditions where every policy struggles.

Comparing the controller against successful reference trajectories under identical conditions surfaces concrete decision-time failure modes: Pensieve selecting bitrates that recent throughput and buffer levels cannot sustain, Sage backing off too sharply after bandwidth drops and reopening too slowly, or Park routing small jobs to slow servers while fast servers sit idle. These comparisons pinpoint systematic decision-time symptoms and provide the corrective signal needed for runtime intervention.

The same counterfactual comparison provides a discrete corrective label from a compact, application-independent set:

$$\mathcal{C}_{\text{adjustment}} := \{\text{ABSTAIN}, \text{BACK_OFF}, \text{PUSH_HARDER}\}. \tag{12}$$

ABSTAIN leaves the controller action unchanged, BACK_OFF indicates that the controller acted too aggressively for the observed state, and PUSH_HARDER indicates that the controller acted too conservatively. When the regret is small, the reference is not reliably better, or the corrective direction is ambiguous, the Counterfactual Analyzer outputs ABSTAIN. The heuristic portfolio thus serves as the source of counterfactual action labels that make runtime protection actionable.

5.2 Learning Protection Rules from Risky States

REGUARD learns threshold-based logic rules that map observable controller state regimes directly to the adjustment labels produced by the Counterfactual Analyzer. To synthesize these rules, REGUARD adapts the rule-learning stage of NetNomos [25] to associate controller state predicates with corrective action labels.

Predicate construction. REGUARD constructs its predicate vocabulary from normal controller behavior by converting tabular state observations into symbolic threshold predicates [26, 32]. It runs the controller on normal traces, computes percentile thresholds for each observable state variable, and forms predicates such as $x_i(s) \geq q^{\text{normal}}_{i,95}$, $x_i(s) \leq q^{\text{normal}}_{i,25}$, or $x_i(s) \geq q^{\text{normal}}_{i,90}$, where $q^{\text{normal}}_{i,p}$ is the pth percentile of feature i on normal traces. Expressing each condition relative to the controller's nominal operating distribution keeps the predicates interpretable across domains [22], capturing high loss or RTT inflation for Sage, low buffer occupancy or high download delay for Pensieve, and load imbalance or large incoming jobs for Park.

Rule synthesis. NetNomos receives the percentile predicates and the Counterfactual Analyzer's adjustment labels and synthesizes implications of the form

$$\phi_1(s_t) \wedge \phi_2(s_t) \wedge \cdots \wedge \phi_k(s_t) \implies \text{adjustment}(s_t) = c, \quad (13)$$

where each $\phi_j(s_t)$ is a percentile threshold predicate on a single state feature and $c \in \mathcal{C}_{\text{adjustment}}$. Each learned rule prescribes a single adjustment whenever its predicate conjunction holds. This simple rule structure keeps the deployed mechanism compact and inspectable, giving operators a clear explanation of when the controller fails and how its action is adjusted.

Iterative refinement. Because an initial rule set may miss secondary failure states that were masked by the most severe initial flaws, REGUARD refines its rules through an iterative dataset-aggregation loop. Each round runs scenario discovery against the *currently protected* controller, surfacing counterexample scenarios that bypass or stress the current rules. REGUARD replays these new scenarios, uses the Counterfactual Analyzer to label the newly visited states, and appends the resulting examples to a cumulative dataset.

In each round, REGUARD re-learns the rule set *from scratch* over the aggregated dataset rather than appending ad hoc patches to the previous rules. Synthesizing a single coherent rule set over all evidence collected so far avoids logical conflicts between old and new clauses, prevents rule explosion, and allows the learner to tighten or relax earlier predicate boundaries as new counterexamples arrive.

This iterative loop directs search capacity toward remaining blind spots while retaining earlier examples of where to intervene and where to abstain, preventing overfitting to the most recent round. As a result, refinement broadens coverage of risky regimes while keeping the deployed rule set compact and consistent.

5.3 Efficient Inference-time Rule Enforcement

REGUARD enforces its learned rules at inference time using fast, memoryless predicate evaluation over the current observable state s_t [23]. At each decision step t, REGUARD evaluates predicates on s_t and adjusts only the controller's proposed action, without tracking full trajectories, running reference policies, or solving an optimization problem online.

This memoryless design matches the standard RL observation interface. RL controllers already summarize recent history, including throughput samples, delay measurements, prior actions, and queue estimates, inside the current state vector s_t. Under the Markov property [13, 29, 40, 43, 52], s_t contains the state features needed for the next decision, so evaluating rules over s_t suffices to detect risky regimes and adjust the proposed action.

Formally, let $\mathcal{R}$ be the learned rule set and let $\pi(s_t)$ be the action proposed by the pretrained controller. REGUARD computes

$$c_t := g_{\mathcal{R}}(s_t) \quad \text{and} \quad a_t^{\text{prot}} := h(\pi(s_t), c_t, s_t),$$

where $g_{\mathcal{R}}$ maps the current state to an adjustment label and h maps that label to an application-specific bounded correction. If $c_t = \text{ABSTAIN}$, then $a_t^{\text{prot}} := \pi(s_t)$. If c_t is BACK_OFF or PUSH_HARDER, then h nudges or masks the action in the direction supported by the Counterfactual Analyzer's labels. The RL controller remains the primary decision maker because REGUARD intervenes only when a learned rule fires.

To minimize runtime latency, REGUARD organizes the conjunctions in $\mathcal{R}$ into a prefix trie [7]. Rules that share predicate prefixes share a common path in the trie, allowing REGUARD to prune entire subtrees as soon as a predicate evaluates to false. Lookup cost therefore scales with the depth of matched predicates rather than the total number of rules.

This architecture creates a sharp computational asymmetry between offline discovery and online protection. While offline scenario search and counterfactual analysis use cluster compute to train outer RL solvers and run multi-policy emulations, online enforcement executes only scalar comparisons over already-computed state features and occupies a few kilobytes of memory. As a result, all protected controllers easily meet their real-time decision deadlines (Fig. 7).

5.4 Implementation

Congestion control. For Sage [59] (with Pensieve and Park protection detailed in Appendix C.2), the protected controller observes the 69-dimensional state vector summarized in Table 2, covering RTT, RTT variation, delivery rate, loss, minimum-RTT ratios, and prior actions. Because Sage outputs a continuous rate-scaling action $a_t \in [a_{\min}, a_{\max}]$, REGUARD

implements its adjustments as bounded nudges of step size $\Delta > 0$ along the primary control dimension:

$$a_t^{\text{prot}} := \begin{cases} a_t, & \text{if } c_t = \text{ABSTAIN}, \\ \text{clip}(a_t - \Delta,\, a_{\min},\, a_{\max}), & \text{if } c_t = \text{BACK_OFF}, \\ \text{clip}(a_t + \Delta,\, a_{\min},\, a_{\max}), & \text{if } c_t = \text{PUSH_HARDER}. \end{cases} \tag{14}$$

This rule-guided nudge preserves Sage as the base controller and adjusts its aggressiveness only when the rules detect a verified risky state.

6 Evaluation

We prototype ReGuard [1] and aim to answer the following evaluation questions through experiments:

- E1. Can ReGuard find network scenarios where a given RL controller incurs a larger performance gap than competing baselines expose?
- E2. Can ReGuard effectively protect a given RL controller without retraining while preserving its nominal performance?
- E3. How does the severity of the scenarios used to learn protection rules affect the effectiveness of ReGuard's protection?
- E4. How much overhead does ReGuard impose on the RL controller during inference?
- E5. Can ReGuard iteratively reduce the performance gap of the most challenging network scenarios to the level of normal scenarios?

6.1 Setup

Network Traces. We leverage diverse network and workload traces (referred to as "Normal" in our experiments) to train and evaluate controllers across intended, diverse operating regimes [31, 55]:

- Sage [59] is evaluated on intra-continental live Internet paths across 16 U.S. cities, intercontinental paths across 13 international servers, and 23 cellular traces gathered in NYC, following prior train-test splits [2, 59].
- Pensieve [38] is evaluated on 375 FCC broadband traces [15] and 425 Norway cellular traces [50] under the train-test split from Genet [56].
- Park [39] is evaluated on 200 traces across 5 workload distributions ("Default", "Original", "RL1", "RL2", and "RL3") [56], split evenly between training and testing.

Baselines for discovering challenging network scenarios.

- **Indago** [9] trains a surrogate failure predictor and uses saliency gradients to search for failure-inducing scenarios.
- **Genet** [56] generates curricula maximizing raw gap-to-baseline (using BBR [12] for CCA, RobustMPC [60] for ABR, and least-load-first for LB).
- **Gilad et al.** [16] directly searches for network conditions that degrade controller performance.
- **Random** uniformly fuzzes network scenario variables across their entire domains (Table 1).

Protection baselines. To protect the RL controller against challenging network scenarios, we compare ReGuard[2] against the following retraining-based approaches:

- **Fine-tuning**: The most direct way to improve an RL policy on a given workload family is to continue training it on that workload. We consider two fine-tuning settings: **(individual)** We fine-tune the RL controller on traces from discovered scenarios in one specific family, and evaluate the resulting policy on held-out test traces of the *same* kind (*e.g.,* fine-tuning on Indago's traces and then testing on Indago's held-out traces). **(all)** We fine-tune the RL controller on traces from *all* scenario families (namely, ReGuard, Indago, Genet, Gilad et al., and Random) and evaluate the resulting policy separately on held-out traces from each individual family.
- **Curriculum learning**: We use Genet [56] to iteratively generate increasingly challenging network scenarios and continue training the RL policy on the discovered traces, thereby exposing it to a progressively more challenging curriculum [5, 20]. For all use cases, we run Genet for 20 rounds, doubling the 10 rounds used in the original paper.

Because Sage [59] is an offline RL method, we fine-tune it following the procedure in Mowgli [3]: we first collect trajectories of the RL policy on the discovered scenarios and then continue training the model offline on those trajectories (testbed details appear in Appendix G).

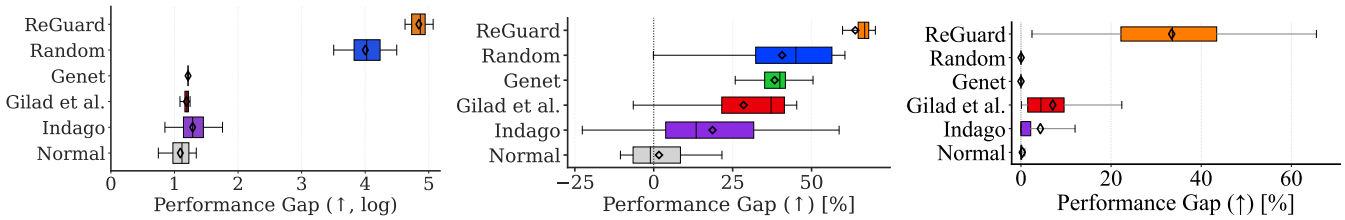
6.2 Results

Finding 1: ReGuard finds the largest performance gaps in all three use cases, certifying severe performance gaps of RL controllers.

ReGuard discovers the largest avoidable performance gaps across all three controllers, outperforming the strongest discovery baselines by 1.6× to 7.0× (E1, Fig. 4). Specifically, ReGuard exposes a mean log QoE gap of 4.84 in Pensieve (7.0× larger than Random on the original scale), a 63.8% performance gap in Sage (1.6× larger than Random), and a 43.5% completion-time gap in Park (6.2× larger than Gilad et al.), compared to baseline gaps of 1.09, 1.7%, and 0.3% on Normal traces. In Park, the wider spread reflects workload heterogeneity across bursty job traces, where ReGuard consistently exposes the largest gap to Least Completion Time across all five workload distributions.

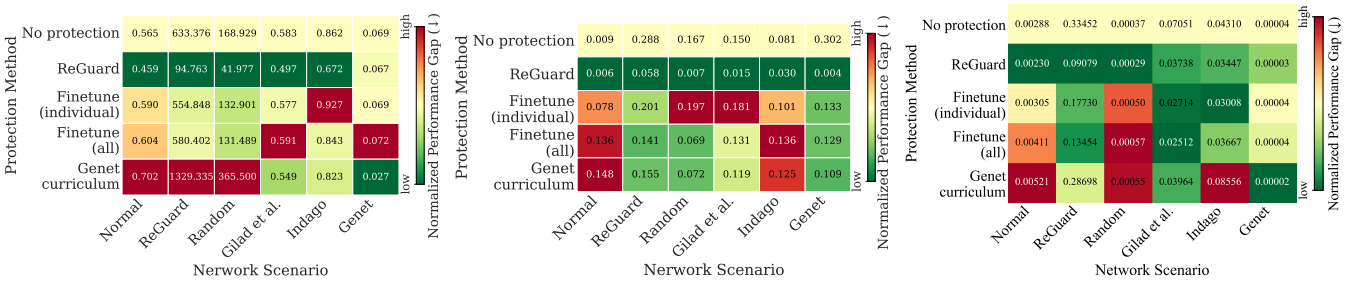
Finding 2: ReGuard, when derived from the largest-gap network scenarios it finds, provides the strongest protection among the compared methods in all three use cases,

[2]The logic rules are always derived from training traces, never from held-out test traces.



(a) Pensieve. (b) Sage. (c) Park.

Figure 4: REGUARD finds the most challenging scenarios in all three use cases. The gap is consistently larger than the strongest baseline and far above the Normal regime, showing that REGUARD exposes large avoidable underperformance rather than marginally harder tests.



(a) Pensieve. (b) Sage. (c) Park.

Figure 5: REGUARD provides the strongest overall protection when derived from REGUARD scenarios, while preserving nominal performance. The results of REGUARD shown here come from its *very first iteration*, and later iterations provide even more protection (see Fig. 8). REGUARD is most effective on the most challenging scenarios, but it also transfers to other, less challenging scenario families, indicating that it captures recurring risky states rather than memorizing patterns from one method.

preserves nominal performance, and remains effective beyond its training scenario family.

In its very first iteration, REGUARD removes 85.0% of the normalized performance gap on Pensieve, 80.0% on Sage, and 79.1% on Park without retraining (E2, Fig. 5). Rules learned from REGUARD's scenarios also transfer across scenario families, removing 75.2% of the gap on Random traces for Pensieve, 90.1%–98.5% on Gilad et al. and Genet traces for Sage, and 68.2% on Gilad et al. traces for Park. On Normal traces, REGUARD preserves and even reduces the residual nominal gap by 18.7%–36.1%, whereas retraining baselines worsen the Normal gap by up to 24.3% on Pensieve, 81.1% on Park, and 15× on Sage (Appendix E). Rules derived from Random scenarios achieve moderate transfer because RL policy flaws span contiguous regions of the state space [16, 49], yet Random discovers much smaller gaps than REGUARD and yields weaker protection overall.

Finding 3: Learning protection rules from scenarios with larger performance gaps produces stronger runtime protection.

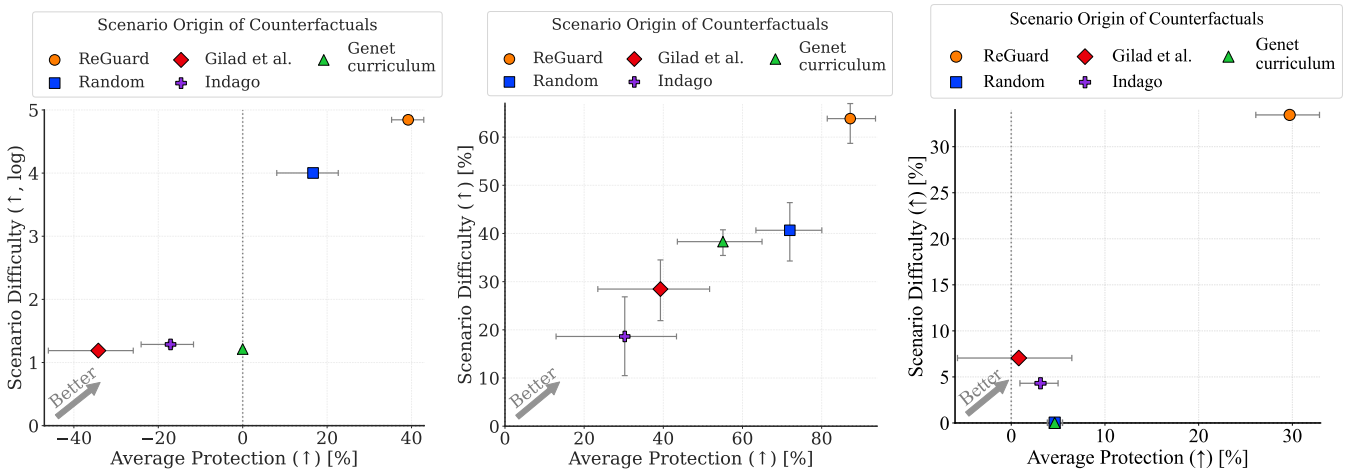
Across all three controllers, REGUARD achieves higher average gap reduction when its rules are synthesized from scenarios with larger discovered regret (E3, Fig. 6). Rules learned from REGUARD's high-regret trajectories consistently outperform rules learned from weaker scenario generators such as Indago, Genet, or Random, confirming that stronger scenario discovery directly produces stronger runtime protection.

Finding 4: REGUARD always fits within the inference time budget.

REGUARD's prefix-trie rule evaluation stays well below every controller's real-time decision deadline across all refinement rounds (E4, Fig. 7). Across iterations 0–3, total decision latency consumes only 11.2%–32.4% of Pensieve's chunk-download budget, 47.6%–50.1% of Sage's 10 ms control interval (compared to 41.4% for unmodified Sage), and 1.9%–6.5% of Park's inter-arrival budget. Adding refinement rounds strengthens protection without increasing inference latency.

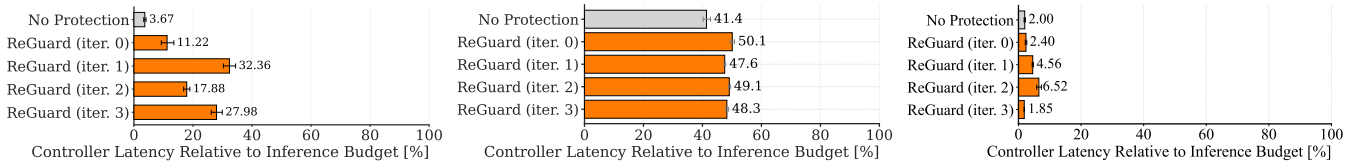
Finding 5: REGUARD does not merely protect against a specific challenging scenario; it fundamentally shrinks the worst-case performance gap.

Running REGUARD's search-and-protect loop for three iterations shrinks the mean discovered gap by 74.4% in Pensieve (from 4.84 to 1.24), 94.1% in Sage (from 63.8% to



(a) Pensieve. (b) Sage. (c) Park.

Figure 6: Learning protection rules from higher-regret scenarios yields stronger runtime protection across all three controllers. Search quality therefore matters directly: discovering severer failures produces more effective protection rules.



(a) Pensieve. (b) Sage. (c) Park.

Figure 7: REGUARD remains well within the online decision budget in all three systems across all refinement iterations. Later iterations strengthen protection, but they do not introduce systematic latency growth.

3.8%), and 92.7% in Park (from 33.5% to 2.4%), closing 93.5%–96.6% of the distance to the Normal regime within two to three rounds (E5, Fig. 8).

7 Analyzing Revealed Performance Failures

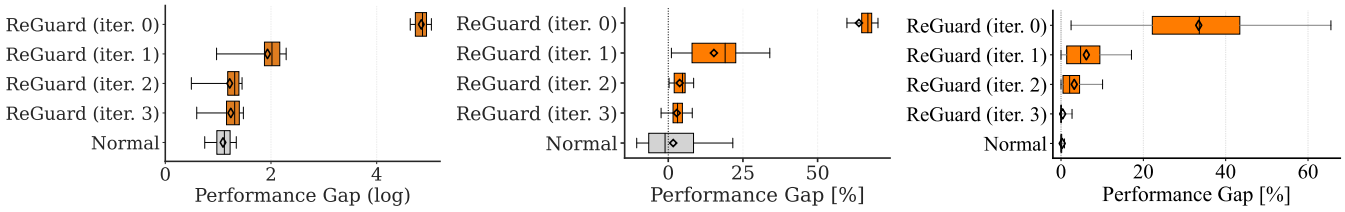
Inspecting the scenarios and logic rules produced by REGUARD reveals the concrete policy flaws behind each controller's worst-case performance gap. In Park, our analysis exposes a systematic dispatch flaw that appears directly in REGUARD's learned rules: when small jobs dominate the arrival stream, Park repeatedly routes them to slow servers even while fast servers sit idle. We focus here on this Park mechanism and defer supporting statistics and the corresponding Pensieve and Sage analyses to Appendices F.1–F.3.

Park's degradation stems from an internal dispatch bias toward slow servers rather than cluster-wide overload. As shown in Fig. 9, when short bursts of large jobs are followed by a stream of small jobs (≤ 0.2 KB) that the fast tier could easily absorb, Park routes 97.3% of those small jobs to slow servers and 86.5% to slow Server 1 alone. Under the exact same arrival stream, Least Completion Time (LCT) routes 81.1% of those small jobs to idle fast servers. Fig. 10 confirms this pattern across the full workload corpus: under REGUARD's scenarios, Park sends 76.3% of all jobs to slow Server 1 (up from 5.6% on normal workloads) even though at least one fast server sits completely idle on 98.0% of decision steps.

REGUARD makes this dispatch flaw explicit by separating risk detection from server-mask recovery using percentile predicates pk calibrated on normal traces (§5). In the rules below, GapToMin_i measures how much server i's current load exceeds the least-loaded server, PostLoad_i denotes server i's projected load after adding the arriving job, and MeanLoad, StdLoad, and TotalLoad summarize cluster-wide queue state. First, a representative risk rule flags incoming small jobs whenever slow Server 1 is backed up while fast Server 9 has spare capacity:

$$\begin{aligned}&(\text{JobSize} \leq \text{p25}) \wedge (\text{GapToMin}_1 > \text{p25}) \\ &\wedge (\text{PostLoad}_9 \leq \text{p25}) \wedge (\text{StdLoad} > \text{p25}) \\ &\implies \text{Risky} = \texttt{true}.\end{aligned} \tag{15}$$

Equation (15) captures the exact failure regime in Fig. 9: an easy job arrives (JobSize $\leq$ p25), load is unevenly distributed across the cluster (StdLoad $>$ p25) with slow Server 1 carrying excess work ($\text{GapToMin}_1 >$ p25), yet fast Server



(a) Pensieve. (b) Sage. (c) Park.

Figure 8: A few search-and-protect iterations are enough to drive the discovered gap close to the Normal regime. Most of the improvement arrives in the first two rounds, after which the remaining challenging cases are much less severe.

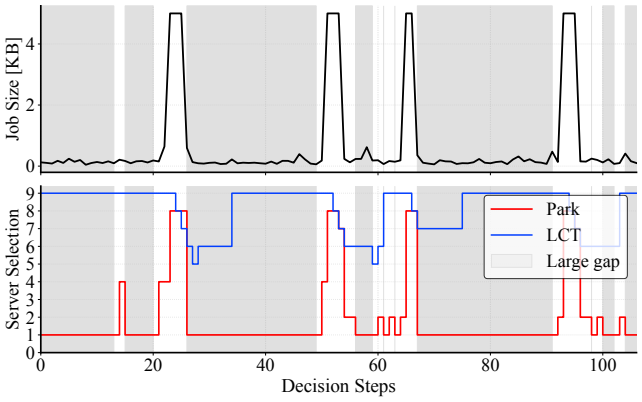


Figure 9: Small-job-heavy scenarios expose Park's bias toward slow servers. Park keeps routing tiny jobs to the slow tier even while fast servers remain idle, which creates prolonged queueing and a large gap to Least Completion Time (LCT).

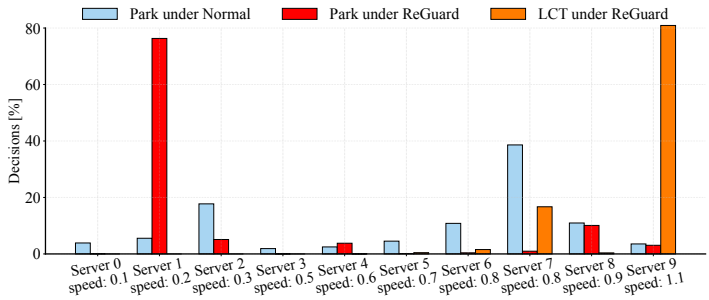


Figure 10: Network scenarios found by REGUARD turn Park's mild preference under nominal conditions into a near-collapse onto slow Server 1. LCT still routes most jobs to the fast tier, which shows that the issue is Park's dispatch policy rather than an inherently bad workload.

9 would remain lightly loaded even after taking the job ($\text{PostLoad}_9 \leq \text{p25}$).

When $\text{Risky} = \texttt{true}$, REGUARD evaluates recovery rules that construct the safe server mask $\mathcal{A}_{\text{safe}}(s_t)$ by reopening underutilized fast servers:

$$(\text{GapToMin}_1 > \text{p25}) \wedge (\text{GapToMin}_9 \leq \text{p25}) \wedge (\text{MeanLoad} \leq \text{p25}) \wedge (\text{TotalLoad} \leq \text{p25}) \implies \text{Allow}_9 = \texttt{true}, \tag{16}$$

$$(\text{Load}_9 \leq \text{p25}) \wedge (\text{PostLoad}_2 \leq \text{p25}) \wedge (\text{MeanLoad} \leq \text{p25}) \wedge (\text{StdLoad} \leq \text{p25}) \implies \text{Allow}_7 = \texttt{true}. \tag{17}$$

Equations (16) and (17) show that REGUARD reopens multiple fast servers (Servers 7 and 9) whenever the cluster is lightly loaded overall but slow Server 1 is disproportionately busy, preventing Park from collapsing onto the slow tier while preserving Park's relative ranking among the allowed fast servers (Appendix F.1).

8 Conclusion

REGUARD discovers worst-case failures in RL-based network controllers by formulating the search for challenging network scenarios as bilevel regret maximization. It protects the controller at inference time using lightweight, interpretable logic rules derived from counterfactual analysis. Across three controllers spanning distinct networking tasks, REGUARD exposes performance gaps up to 6× larger than prior methods and closes up to 85% of those gaps within the first refinement iteration, without retraining and without sacrificing nominal performance.

Acknowledgments

We thank the anonymous reviewers for their insightful feedback and comments, which meaningfully improved this paper. This work was supported by the National Science Foundation (NSF) through Grants CNS-2442625 and CNS-2319442 and Office of Naval Research (ONR) through Grant N000142612552.

References

[1] ReGuard codebase. https://github.com/HongyuHe/ReGuard.

[2] Soheil Abbasloo, Chen-Yu Yen, and H Jonathan Chao. Classic meets modern: A pragmatic learning-based congestion control for the internet. In *Proceedings of the Annual conference of the ACM Special Interest Group on Data Communication on the applications, technologies, architectures, and protocols for computer communication*, pages 632–647, 2020.

[3] Neil Agarwal, Rui Pan, Francis Y. Yan, and Ravi Netravali. Mowgli: Passively learned rate control for real-time video. In *22nd USENIX Symposium on Networked Systems Design and Implementation (NSDI 25)*, pages 579–594, 2025. URL https://www.usenix.org/conference/nsdi25/presentation/agarwal.

[4] Sohel Ahmed, Jonathan Sentosa, Kevin Zhang, Yoav Lebendiker, Michael Shnaiderman, Tomer Gilad, Nathan Jay, Brighten Godfrey, and Michael Schapira. AdvNet: Revealing performance issues in network protocols by searching for adversarial environments. In *Proceedings of the 2026 ACM International Conference on emerging Networking EXperiments and Technologies (CoNEXT '26)*, 2026.

[5] Ilge Akkaya, Marcin Andrychowicz, Maciek Chociej, Mateusz Litwin, Bob McGrew, Arthur Petron, Alex Paino, Matthias Plappert, Glenn Powell, Raphael Ribas, et al. Solving rubik's cube with a robot hand. *arXiv preprint arXiv:1910.07113*, 2019.

[6] Venkat Arun, Mina Tahmasbi Arashloo, Ahmed Saeed, Mohammad Alizadeh, and Hari Balakrishnan. Toward formally verifying congestion control behavior. In *Proceedings of the 2021 ACM SIGCOMM 2021 Conference*, pages 1–16, 2021.

[7] Ryan Beckett, Francis Y. Yan, Raghunadha Reddy Pocha, Vineesh V. Raj, Ayyub Shaik, and Siva Kesava Reddy Kakarla. Concord: Learning network configuration contracts. In *Proceedings of the 21st European Conference on Computer Systems (EuroSys '26)*, page 18, Edinburgh, Scotland, UK, April 2026. ACM. ISBN 979-8-4007-2212-7/26/04. doi: 10.1145/3767295.3769338.

[8] Matan Ben-Tov, Daniel Deutch, Nave Frost, and Mahmood Sharif. Cafa: Cost-aware, feasible attacks with database constraints against neural tabular classifiers. In *2024 IEEE Symposium on Security and Privacy (SP)*, pages 1345–1364. IEEE, 2024.

[9] Matteo Biagiola and Paolo Tonella. Testing of deep reinforcement learning agents with surrogate models. *ACM Transactions on Software Engineering and Methodology*, 33(3):73:1–73:33, 2024. doi: 10.1145/3631970.

[10] Stephen Boyd and Lieven Vandenberghe. *Convex optimization*. Cambridge university press, 2004.

[11] Neal Cardwell, Yuchung Cheng, Lawrence Brakmo, Matt Mathis, Barath Raghavan, Nandita Dukkipati, Hsiao-keng Jerry Chu, Andreas Terzis, and Tom Herbert. packetdrill: Scriptable network stack testing, from sockets to packet wire. In *2013 USENIX Annual Technical Conference (USENIX ATC 13)*, pages 213–218, 2013.

[12] Neal Cardwell, Yuchung Cheng, C Stephen Gunn, Soheil Hassas Yeganeh, and Van Jacobson. Bbr: Congestion-based congestion control. *Communications of the ACM*, 60(2):58–66, 2017.

[13] Mo Dong, Tong Meng, Doron Zarchy, Engin Arslan, Yossi Gilad, Brighten Godfrey, and Michael Schapira. PCC vivace: Online-learning congestion control. In *15th USENIX Symposium on Networked Systems Design and Implementation (NSDI 18)*, pages 343–356, 2018.

[14] Tomer Eliyahu, Yafim Kazak, Guy Katz, and Michael Schapira. Verifying learning-augmented systems. In *Proceedings of the 2021 ACM SIGCOMM 2021 Conference*, pages 305–318, 2021.

[15] Federal Communications Commission. Measuring broadband america. https://www.fcc.gov/general/measuring-broadband-america, 2016.

[16] Tomer Gilad, Nathan H. Jay, Michael Shnaiderman, Brighten Godfrey, and Michael Schapira. Robustifying network protocols with adversarial examples. In *Proceedings of the 18th ACM Workshop on Hot Topics in Networks*, pages 85–92. ACM, 2019. ISBN 978-1-4503-7020-2. doi: 10.1145/3365609.3365862. URL https://dl.acm.org/doi/10.1145/3365609.3365862.

[17] Allen A Goldstein. Convex programming in hilbert space. *Bulletin of the American Mathematical Society*, 70(5):709–710, 1964.

[18] Fengchen Gong, Divya Raghunathan, Aarti Gupta, and Maria Apostolaki. Towards integrating formal methods into ml-based systems for networking. In *Proceedings of the 22nd ACM Workshop on Hot Topics in Networks*, pages 48–55, 2023. doi: 10.1145/3626111.3628188. URL https://doi.org/10.1145/3626111.3628188.

[19] Fengchen Gong, Divya Raghunathan, Aarti Gupta, and Maria Apostolaki. Zoom2Net: Constrained network

telemetry imputation. In *Proceedings of the ACM SIGCOMM 2024 Conference*, pages 764–777, 2024. doi: 10.1145/3651890.3672225. URL https://doi.org/10.1145/3651890.3672225.

[20] Alex Graves, Marc G Bellemare, Jacob Menick, Remi Munos, and Koray Kavukcuoglu. Automated curriculum learning for neural networks. In *international conference on machine learning*, pages 1311–1320. Pmlr, 2017.

[21] Hongyu Hè. FPGA-based SmartNIC for distributed machine learning. Master's thesis, ETH Zurich, 2024.

[22] Hongyu Hè. PerOS: Personalized self-adapting operating systems in the cloud. *arXiv preprint arXiv:2404.00057*, 2024.

[23] Hongyu Hè and Maria Apostolaki. Just-in-time logic enforcement: A new paradigm of combining statistical and symbolic reasoning for network management. In *Proceedings of the 24th ACM Workshop on Hot Topics in Networks (HotNets 25)*, pages 184–192, 2025. doi: 10.1145/3772356.3772406. URL https://doi.org/10.1145/3772356.3772406.

[24] Hongyu Hè and Maria Apostolaki. Let AI agents translate networks, not reason about them. *arXiv preprint arXiv:2607.22947*, 2026. doi: 10.48550/arXiv.2607.22947. URL https://arxiv.org/abs/2607.22947.

[25] Hongyu Hè, Minhao Jin, and Maria Apostolaki. Making logic a First-Class citizen in generative ML for networking. In *23rd USENIX Symposium on Networked Systems Design and Implementation (NSDI 26)*, pages 801–824. USENIX Association, 2026. URL https://www.usenix.org/conference/nsdi26/presentation/he.

[26] Hongyu Hè, Alexander Krentsel, Sylvia Ratnasamy, and Maria Apostolaki. Invariant discovery for networked systems. In *ACM Workshop on Hot Topics in Networks (HotNets '26)*, 2026. URL https://arxiv.org/abs/2607.22944. To appear.

[27] Mehdi Hosseinzadeh, Karthick Shankar, Maria Apostolaki, Jay Ramachandran, Steven E Adams, Vyas Sekar, and Bruno Sinopoli. Cane: A cascade control approach for network-assisted video qoe management. *IEEE Transactions on Control Systems Technology*, 2023.

[28] Arthur S. Jacobs, Roman Beltiukov, Walter Willinger, Ronaldo A. Ferreira, Arpit Gupta, and Lisandro Z. Granville. AI/ML for Network Security: The Emperor has no Clothes. In *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security (CCS '22)*, Los Angeles, CA, USA, 2022. ACM. doi: 10.1145/3548606.3560609. URL https://trusteeml.github.io/.

[29] Nathan Jay, Noga Rotman, Brighten Godfrey, Michael Schapira, and Aviv Tamar. A deep reinforcement learning perspective on internet congestion control. In *International Conference on Machine Learning (ICML)*, pages 3050–3059. PMLR, 2019.

[30] Minhao Jin and Maria Apostolaki. Robustifying ml-powered network classifiers with pants. In *34th USENIX Security Symposium (USENIX Security 25)*, 2025.

[31] Minhao Jin, Hongyu Hè, and Maria Apostolaki. Cross-flow correlations survive synthesis: Measuring source-level privacy leakage in synthetic network traces. In *Network and Distributed System Security Symposium (NDSS 27)*, 2027.

[32] Benno Kruit, Hongyu Hè, and Jacopo Urbani. Tab2Know: Building a knowledge base from tables in scientific papers. In *Proceedings of the 19th International Semantic Web Conference (ISWC 20)*, pages 349–365. Springer, 2020.

[33] Ananya Kumar, Aditi Raghunathan, Robbie Matthew Jones, Tengyu Ma, and Percy Liang. Fine-tuning can distort pretrained features and underperform out-of-distribution. In *International Conference on Learning Representations*, 2022. URL https://openreview.net/forum?id=UYneFzXSJWh.

[34] Aviral Kumar, Justin Fu, Matthew Soh, George Tucker, and Sergey Levine. Stabilizing off-policy q-learning via bootstrapping error reduction. *Advances in neural information processing systems*, 32, 2019.

[35] Evgenii S Levitin and Boris T Polyak. Constrained minimization methods. *USSR Computational Mathematics and Mathematical Physics*, 6(5):1–50, 1966.

[36] Qin-Wen Luo, Ming-Kun Xie, Ye-Wen Wang, and Sheng-Jun Huang. Optimistic critic reconstruction and constrained fine-tuning for general offline-to-online rl. In *Advances in Neural Information Processing Systems*, volume 37, 2024. doi: 10.52202/079017-3435.

[37] Aleksander Madry, Aleksandar Makelov, Ludwig Schmidt, Dimitris Tsipras, and Adrian Vladu. Towards deep learning models resistant to adversarial attacks. In *International Conference on Learning Representations*, 2018. URL https://openreview.net.

[38] Hongzi Mao, Ravi Netravali, and Mohammad Alizadeh. Neural adaptive video streaming with pensieve. In *Proceedings of the Conference of the ACM Special Interest Group on Data Communication*, pages 197–210. ACM, 2017. ISBN 978-1-4503-4653-5. doi:

10.1145/3098822.3098843. URL https://dl.acm.org/doi/10.1145/3098822.3098843.

[39] Hongzi Mao, Parimarjan Negi, Akshay Narayan, Hanrui Wang, Jiacheng Yang, Haonan Wang, Ryan Marcus, Mehrdad Khani Shirkoohi, Songtao He, Vikram Nathan, et al. Park: An open platform for learning-augmented computer systems. *Advances in Neural Information Processing Systems*, 32, 2019.

[40] Andrew Kachites McCallum. *Reinforcement learning with hidden states*. PhD thesis, University of Rochester, 1995.

[41] Zili Meng, Minhu Wang, Jiasong Bai, Mingwei Xu, Hongzi Mao, and Hongxin Hu. Interpreting deep learning-based networking systems. In *Proceedings of the Annual conference of the ACM Special Interest Group on Data Communication on the applications, technologies, architectures, and protocols for computer communication*, SIGCOMM '20, pages 154–171. Association for Computing Machinery, 2020. ISBN 978-1-4503-7955-7. doi: 10.1145/3387514.3405859. URL https://dl.acm.org/doi/10.1145/3387514.3405859.

[42] Zili Meng, Nirav Atre, Bochun Zhang, Mingwei Xu, Justine Sherry, and Maria Apostolaki. Confucius: Adapting home routers to congestion control's reactions for consistent low latency. In *IEEE INFOCOM 2026*, 2026. URL https://netsyn.princeton.edu/document/791.

[43] Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Andrei A Rusu, Joel Veness, Marc G Bellemare, Alex Graves, Martin Riedmiller, Andreas K Fidjeland, Georg Ostrovski, et al. Human-level control through deep reinforcement learning. *Nature*, 518(7540):529–533, 2015.

[44] Pooria Namyar, Behnaz Arzani, Ryan Beckett, Santiago Segarra, Himanshu Raj, Umesh Krishnaswamy, Ramesh Govindan, and Srikanth Kandula. Finding adversarial inputs for heuristics using multi-level optimization. In *21st USENIX Symposium on Networked Systems Design and Implementation (NSDI 24)*, pages 927–949, 2024.

[45] Ravi Netravali, Anirudh Sivaraman, Somak Das, Ameesh Goyal, Keith Winstein, James Mickens, and Hari Balakrishnan. Mahimahi: accurate {Record-and-Replay} for {HTTP}. In *2015 USENIX Annual Technical Conference (USENIX ATC 15)*, pages 417–429, 2015.

[46] Lorenzo Pappone, Alessio Sacco, and Flavio Esposito. Mutant: Learning congestion control from existing protocols via online reinforcement learning. In *22nd USENIX Symposium on Networked Systems Design and Implementation (NSDI 25)*, pages 1507–1522, 2025.

[47] Fabio Pierazzi, Feargus Pendlebury, Jacopo Cortellazzi, and Lorenzo Cavallaro. Intriguing properties of adversarial ml attacks in the problem space. In *2020 IEEE Symposium on Security and Privacy (SP)*, pages 1332–1349, 2020. doi: 10.1109/SP40000.2020.00073.

[48] Divya Raghunathan, Maria Apostolaki, and Aarti Gupta. A layered formal methods approach to answering queue-related queries. In *22nd USENIX Symposium on Networked Systems Design and Implementation (NSDI 25)*, pages 1289–1304. USENIX Association, 2025. URL https://www.usenix.org/conference/nsdi25/presentation/raghunathan.

[49] Siddhant Ray, Alexander Dietmüller, and Laurent Vanbever. CC-Fuzz: Fuzzing network protocol implementations for congestion control vulnerabilities. In *Proceedings of the 21st ACM Workshop on Hot Topics in Networks (HotNets '22)*, pages 160–167, 2022.

[50] Haakon Riiser, Paul Vigmostad, Carsten Griwodz, and Pål Halvorsen. Commute path bandwidth traces from 3G networks in Norway. In *Proceedings of the 4th ACM Multimedia Systems Conference (MMSys '13)*, pages 114–118, 2013.

[51] Divyanshu Saxena, Gaurav Vipat, Jiaxin Lin, Jingbo Wang, Isil Dillig, Sanjay Shakkottai, and Aditya Akella. Towards performance robustness for microservices. In *Proceedings of the 23rd USENIX Symposium on Networked Systems Design and Implementation (NSDI '26)*, Renton, WA, USA, May 2026. USENIX Association. Presented May 4–6, 2026.

[52] Richard S Sutton and Andrew G Barto. *Reinforcement learning: An introduction*. MIT press, 2018.

[53] Brian Thompson, Jeremy Gwinnup, Huda Khayrallah, Kevin Duh, and Philipp Koehn. Overcoming catastrophic forgetting during domain adaptation of neural machine translation. In Jill Burstein, Christy Doran, and Thamar Solorio, editors, *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, pages 2062–2068, Minneapolis, Minnesota, June 2019. Association for Computational Linguistics. doi: 10.18653/v1/N19-1209. URL https://aclanthology.org/N19-1209/.

[54] Dimitris Tsipras, Shibani Santurkar, Logan Engstrom, Alexander Turner, and Aleksander Madry. Robustness may be at odds with accuracy. In *International Conference on Learning Representations*, 2019. URL https://openreview.net/forum?id=SyxAb30cY7.

[55] Dmitrii Ustiugov, Dohyun Park, Lazar Cvetković, Mihajlo Djokic, Hongyu Hè, Boris Grot, and Ana Klimovic. Enabling in-vitro serverless systems research. In *Proceedings of the 4th Workshop on Resource Disaggregation and Serverless (WORDS 23)*. ACM, 2023.

[56] Zhengxu Xia, Yajie Zhou, Francis Y Yan, and Junchen Jiang. Genet: Automatic curriculum generation for learning adaptation in networking. In *Proceedings of the ACM SIGCOMM 2022 Conference*, pages 397–413, 2022.

[57] Francis Y Yan, Hudson Ayers, Chenzhi Zhu, Sadjad Fouladi, James Hong, Keyi Zhang, Philip Levis, and Keith Winstein. Learning in situ: a randomized experiment in video streaming. In *17th USENIX Symposium on Networked Systems Design and Implementation (NSDI 20)*, pages 495–511, 2020.

[58] Chenxi Yang, Divyanshu Saxena, Rohit Dwivedula, Kshiteej Mahajan, Swarat Chaudhuri, and Aditya Akella. Canopy: Property-driven learning for congestion control. In *Proceedings of the 21st European Conference on Computer Systems (EuroSys '26)*, 2026. doi: 10.1145/3767295.3769362. URL https://doi.org/10.1145/3767295.3769362.

[59] Chen-Yu Yen, Soheil Abbasloo, and H. Jonathan Chao. Computers can learn from the heuristic designs and master internet congestion control. In *Proceedings of the ACM SIGCOMM 2023 Conference*, ACM SIGCOMM '23, page 255–274, New York, NY, USA, 2023. Association for Computing Machinery. ISBN 9798400702365. doi: 10.1145/3603269.3604838. URL https://doi.org/10.1145/3603269.3604838.

[60] Xiaoqi Yin, Abhishek Jindal, Vyas Sekar, and Bruno Sinopoli. A control-theoretic approach for dynamic adaptive video streaming over HTTP. In *Proceedings of the 2015 ACM Conference on Special Interest Group on Data Communication*, pages 325–338. ACM, 2015. ISBN 978-1-4503-3542-3. doi: 10.1145/2785956.2787486. URL https://dl.acm.org/doi/10.1145/2785956.2787486.

[61] Anchengcheng Zhou, Carter Costic, Hongyu Hè, Ahmad Ghalayini, Abdul Kabbani, and Maria Apostolaki. Mitigating inter-datacenter incast with a proxy: The shortest path is not necessarily the fastest. In *Proceedings of the 24th ACM Workshop on Hot Topics in Networks (HotNets '25)*, pages 344–353. Association for Computing Machinery, 2025. doi: 10.1145/3772356.3772410. URL https://doi.org/10.1145/3772356.3772410.

[62] Anchengcheng Zhou, Joshua Lau, P. Brighten Godfrey, and Maria Apostolaki. Controlling arbitrary internet queues with Titrate. In *23rd USENIX Symposium on Networked Systems Design and Implementation (NSDI 26)*, pages 1453–1467. USENIX Association, 2026. URL https://www.usenix.org/conference/nsdi26/presentation/zhou-titrate.

A Proofs for Regret Guarantees

This appendix gives the detailed assumptions, intermediate lemmas, and full proofs for the regret guarantees in Section 4.2. Our goal is to make explicit what is guaranteed by the formulation, what approximation errors enter the final bound, and how those errors propagate through the argument.

A.1 Setup and Notation

We begin by restating the exact and approximate regret objectives used in the main text. For every feasible network scenario $e \in \mathcal{E}$, define

$$R(e) := J(\pi_e^\star; e) - J(\pi; e), \qquad \hat{R}(e) := J(\hat{\pi}_e^\star; e) - J(\pi; e). \tag{18}$$

Here:

- $R(e)$ is the *exact* regret objective, where the comparator $\pi_e^\star$ is an exact optimizer within the reference policy class:

$$\pi_e^\star \in \arg\max_{\pi' \in \Pi} J(\pi'; e).$$

- $\hat{R}(e)$ is the *approximate* regret objective, where the comparator $\hat{\pi}_e^\star \in \Pi$ is the practical reference policy used by the implementation.

The exact bilevel target in the main text is

$$e^\star \in \arg\max_{e \in \mathcal{E}} R(e), \tag{19}$$

while the implemented procedure approximately solves

$$\hat{e} \in \arg\max_{e \in \mathcal{E}} \hat{R}(e). \tag{20}$$

The proofs below establish three facts. First, the outer solver returns a near-optimal scenario for the approximate objective. Second, the approximate objective is a pointwise lower bound on the exact objective. Third, combining these two facts yields the final exact-regret guarantee in Theorem 1.

A.2 Assumptions

We restate the assumptions from the main text for completeness.

A1: The feasible set $\mathcal{E}$ is non-empty, and both Eqns. (3) and (5) admit optimizers.

A2: For every $e \in \mathcal{E}$, the rewards $J(\pi_e^\star; e)$, $J(\hat{\pi}_e^\star; e)$, and $J(\pi; e)$ are finite.

A3: The outer RL solver returns an ε-optimal solution $\tilde{e} \in \mathcal{E}$ to Eqn. (5), namely

$$\max_{e \in \mathcal{E}} \hat{R}(e) - \hat{R}(\tilde{e}) \le \varepsilon. \tag{21}$$

A4: The approximate reference satisfies

$$J(\pi_e^\star; e) - J(\hat{\pi}_e^\star; e) \le \delta(e) \tag{22}$$

for every $e \in \mathcal{E}$, where $\delta(e) \ge 0$.

These assumptions are mild in the settings studied in the paper. A1 says that the operator-defined feasible family of network scenarios is nontrivial and that both optimization problems are well posed. A2 excludes degenerate cases in which the trajectory reward is undefined or unbounded. A3 captures the quality of the outer solver. It does not require exact optimization, only that the scenario returned by the solver be within ε of the best achievable approximate-regret score. A4 captures the quality of the inner reference policy. It says that the approximate reference does not fall more than $\delta(e)$ below the exact best reference in scenario e.

The final theorem separates these two error sources cleanly. The term ε accounts for suboptimality in the outer search over scenarios. The term $\delta(e^\star)$ accounts for the error of the practical reference at an exact worst-case scenario. This separation is useful because the two terms arise from different components of the system and can be improved independently.

A.3 Auxiliary Observations

We first record two elementary but useful observations that will be used repeatedly in the proofs.

Lemma 2. *For every feasible scenario $e \in \mathcal{E}$,*

$$J(\pi_e^\star; e) \ge J(\hat{\pi}_e^\star; e). \tag{23}$$

Proof. Fix any $e \in \mathcal{E}$. By definition, $\pi_e^\star$ is an optimizer of $J(\pi'; e)$ over the reference class Π. Since $\hat{\pi}_e^\star \in \Pi$, the optimality of $\pi_e^\star$ implies

$$J(\pi_e^\star; e) \ge J(\hat{\pi}_e^\star; e).$$

This proves the claim. □

Lemma 3. *For every feasible scenario $e \in \mathcal{E}$,*

$$R(e) - \hat{R}(e) = J(\pi_e^\star; e) - J(\hat{\pi}_e^\star; e). \tag{24}$$

Proof. By the definitions in Eqn. (18),

$$R(e) = J(\pi_e^\star; e) - J(\pi; e), \qquad \hat{R}(e) = J(\hat{\pi}_e^\star; e) - J(\pi; e).$$

Subtracting the second identity from the first cancels the common term $J(\pi; e)$ and yields

$$R(e) - \hat{R}(e) = J(\pi_e^\star; e) - J(\hat{\pi}_e^\star; e).$$

□

The next observation makes explicit that the approximate objective is always conservative.

Lemma 4. *For every feasible scenario $e \in \mathcal{E}$,*

$$\hat{R}(e) \le R(e). \tag{25}$$

Proof. Fix any $e \in \mathcal{E}$. By Lemma 2,

$$J(\pi_e^\star; e) \ge J(\hat{\pi}_e^\star; e).$$

Subtracting the common term $J(\pi; e)$ from both sides preserves the inequality, giving

$$J(\pi_e^\star; e) - J(\pi; e) \ge J(\hat{\pi}_e^\star; e) - J(\pi; e).$$

By Eqn. (18), this is exactly

$$R(e) \ge \hat{R}(e).$$

□

Lemma 4 is conceptually important. It says that replacing the exact reference policy with the practical reference can only decrease the regret score. In other words, the approximate regret objective used by the implementation is a pointwise lower bound on the ideal objective of interest.

A.4 Outer-search Guarantee

We now formalize the guarantee provided by the outer RL solver on the approximate objective.

Lemma 5. *Under A1 and A3, the scenario $\tilde{e}$ returned by the outer solver satisfies*

$$\max_{e \in \mathcal{E}} \hat{R}(e) - \hat{R}(\tilde{e}) \le \varepsilon. \tag{26}$$

Equivalently,

$$\hat{R}(\tilde{e}) \ge \max_{e \in \mathcal{E}} \hat{R}(e) - \varepsilon. \tag{27}$$

Proof. By A1, the approximate outer problem in Eqn. (5) admits an optimizer. Let $\hat{e}^\star \in \mathcal{E}$ be such that

$$\hat{R}(\hat{e}^\star) = \max_{e \in \mathcal{E}} \hat{R}(e). \tag{28}$$

By A3, the outer solver returns a scenario $\tilde{e} \in \mathcal{E}$ whose approximate objective value is within ε of the optimum:

$$\max_{e \in \mathcal{E}} \hat{R}(e) - \hat{R}(\tilde{e}) \le \varepsilon. \tag{29}$$

This is exactly Eqn. (26). Rearranging Eqn. (26) gives

$$\hat{R}(\tilde{e}) \ge \max_{e \in \mathcal{E}} \hat{R}(e) - \varepsilon,$$

which is Eqn. (27). □

Although straightforward, Lemma 5 plays an important role. It tells us that the outer solver returns a scenario whose *approximate* regret score is nearly the largest possible over the feasible family. This is the first half of the final argument. The second half is to connect approximate regret to exact regret.

A.5 Pointwise Approximation Guarantee

We next show that the approximate regret is a pointwise lower bound on exact regret and that the gap is controlled exactly by the inner-reference error.

Lemma 6. *Under A2 and A4, every $e \in \mathcal{E}$ satisfies*

$$0 \le R(e) - \hat{R}(e) \le \delta(e). \tag{30}$$

Proof. Fix any $e \in \mathcal{E}$. We prove the lower and upper bounds separately.

Lower bound. By Lemma 3,

$$R(e) - \hat{R}(e) = J(\pi_e^\star; e) - J(\hat{\pi}_e^\star; e).$$

By Lemma 2,

$$J(\pi_e^\star; e) - J(\hat{\pi}_e^\star; e) \ge 0.$$

Combining the two equations yields

$$R(e) - \hat{R}(e) \ge 0. \tag{31}$$

Upper bound. Again by Lemma 3,

$$R(e) - \hat{R}(e) = J(\pi_e^\star; e) - J(\hat{\pi}_e^\star; e).$$

A4 states exactly that

$$J(\pi_e^\star; e) - J(\hat{\pi}_e^\star; e) \le \delta(e).$$

Therefore,

$$R(e) - \hat{R}(e) \le \delta(e). \tag{32}$$

Combining Eqns. (31) and (32) proves

$$0 \le R(e) - \hat{R}(e) \le \delta(e).$$

□

Lemma 6 is the precise statement that the approximate objective is conservative. At every scenario e, the exact regret is at least as large as the approximate regret. Furthermore, the amount by which the approximate objective can underestimate the exact objective is no larger than the inner-reference error $\delta(e)$. This pointwise control is what allows us to lift an approximate-optimality guarantee for the outer solver into an exact-regret certificate.

A.6 Exact-regret Guarantee

We now prove the main guarantee from the paper.

Proof of Theorem 1. By A1, the exact outer problem in Eqn. (3) admits an optimizer. Let

$$e^\star \in \arg\max_{e \in \mathcal{E}} R(e). \tag{33}$$

Then by definition,

$$R(e^\star) = \max_{e\in\mathcal{E}} R(e). \tag{34}$$

Our goal is to lower-bound $R(\tilde{e})$ in terms of $R(e^\star)$ and then substitute Eqn. (34).

Step 1: compare the returned scenario to the best approximate-regret scenario. By Lemma 5,

$$\hat{R}(\tilde{e}) \geq \max_{e\in\mathcal{E}} \hat{R}(e) - \varepsilon. \tag{35}$$

Since $e^\star \in \mathcal{E}$, the maximum over $\mathcal{E}$ is at least the value at $e^\star$. Therefore,

$$\hat{R}(\tilde{e}) \geq \hat{R}(e^\star) - \varepsilon. \tag{36}$$

Step 2: relate exact and approximate regret at the returned scenario. Applying Lemma 6 to the scenario $\tilde{e}$ gives

$$R(\tilde{e}) \geq \hat{R}(\tilde{e}). \tag{37}$$

Step 3: relate exact and approximate regret at the exact worst-case scenario. Applying Lemma 6 to the scenario $e^\star$ gives

$$0 \leq R(e^\star) - \hat{R}(e^\star) \leq \delta(e^\star).$$

Rearranging the upper bound yields

$$\hat{R}(e^\star) \geq R(e^\star) - \delta(e^\star). \tag{38}$$

Step 4: chain the inequalities. Starting from Eqn. (37) and then using Eqns. (36) and (38), we obtain

$$\begin{aligned} R(\tilde{e}) &\geq \hat{R}(\tilde{e}) && (39)\\ &\geq \hat{R}(e^\star) - \varepsilon && (40)\\ &\geq R(e^\star) - \delta(e^\star) - \varepsilon. && (41) \end{aligned}$$

Substituting Eqn. (34) into Eqn. (41) gives

$$R(\tilde{e}) \geq \max_{e\in\mathcal{E}} R(e) - \varepsilon - \delta(e^\star). \tag{42}$$

This is exactly Eqn. (7) in the main text.

Finally, rearranging Eqn. (42) yields

$$\max_{e\in\mathcal{E}} R(e) - R(\tilde{e}) \leq \varepsilon + \delta(e^\star), \tag{43}$$

which is equivalent to Eqn. (7) in the main text. This completes the proof. □

A.7 Interpretation of the Final Bound

Theorem 1 has a clean interpretation.

First, the regret achieved by the returned scenario, $R(\tilde{e})$, is itself a *certificate value*. It lower-bounds the worst-case exact regret over the feasible family up to the additive error $\varepsilon + \delta(e^\star)$. Thus, if $R(\tilde{e})$ is large, then the controller provably has a substantial avoidable performance gap in the searched regime.

Second, the theorem isolates the only two sources of looseness in the certificate:

- ε is the suboptimality of the outer search over scenarios. It is controlled by the quality of the RL-based outer solver.
- $\delta(e^\star)$ is the suboptimality of the practical reference at an exact worst-case scenario. It is controlled by the quality of the inner reference policy.

Third, the dependence on $\delta(e^\star)$ rather than a worst-case uniform bound over all $e \in \mathcal{E}$ is meaningful. The theorem only needs the inner-reference quality at an exact optimizer of the true objective. If the practical reference is especially accurate near the most informative failure scenarios, then the final certificate can still be tight even if the reference is less accurate elsewhere.

A.8 A Uniform-error Corollary

In some settings, it is convenient to summarize inner-reference quality by a uniform bound. The next corollary makes this explicit.

Corollary 7. *Suppose the assumptions of Theorem 1 hold, and in addition there exists a constant $\bar{\delta} \geq 0$ such that*

$$\delta(e) \leq \bar{\delta}, \qquad \forall e \in \mathcal{E}. \tag{44}$$

Then the returned scenario $\tilde{e}$ satisfies

$$\max_{e\in\mathcal{E}} R(e) - R(\tilde{e}) \leq \varepsilon + \bar{\delta}, \tag{45}$$

or equivalently,

$$R(\tilde{e}) \geq \max_{e\in\mathcal{E}} R(e) - \varepsilon - \bar{\delta}. \tag{46}$$

Proof. By Theorem 1,

$$\max_{e\in\mathcal{E}} R(e) - R(\tilde{e}) \leq \varepsilon + \delta(e^\star).$$

Under Eqn. (44), we have $\delta(e^\star) \leq \bar{\delta}$. Substituting this into the previous inequality yields Eqn. (45). Rearranging gives Eqn. (46). □

Corollary 7 is weaker than Theorem 1, but sometimes easier to state. It says that if the practical reference is uniformly within $\bar{\delta}$ of the exact reference everywhere, then the final exact-regret certificate loses at most $\varepsilon + \bar{\delta}$ compared to the ideal exact worst case.

A.9 What the Guarantees Do and Do Not Say

For completeness, we summarize the scope of the above results.

What the guarantees establish. The proofs show that, under A1–A4, the returned scenario $\tilde{e}$ certifies a large avoidable performance gap for the pretrained controller. More precisely, the exact regret value $R(\tilde{e})$ is guaranteed to be close to the largest exact regret achievable over the feasible search space.

The approximation error is explicit and decomposes additively into outer-search error and inner-reference error.

What the guarantees do not establish. The guarantees do not claim that the returned scenario is unique. They also do not claim that a particular nonconvex RL algorithm always meets A3. Instead, the theorem is conditional: *if* the outer solver is ε-optimal for the implemented approximate objective, and *if* the practical reference is within $\delta(e)$ of the exact reference, *then* the returned scenario is a near-tight certificate for the exact worst-case regret over the feasible family.

Why the conditional form is still useful. This conditional structure matches how systems are analyzed in practice. The theoretical formulation makes precise what quantity the method is targeting. The theorem then shows exactly how optimization and approximation errors degrade the final certificate. As the outer solver improves or the practical reference becomes stronger, the guarantee strengthens immediately and transparently.

B Use Cases

This appendix summarizes the three controller settings used throughout the paper.

Adaptive bitrate streaming. An adaptive bitrate controller chooses the bitrate of the next video chunk at each chunk boundary. Its decision is based on the recent throughput history, the current playback buffer, and information about upcoming chunks. Pensieve [38] is an RL-based instance of this setting. Its objective is to keep video quality high over the full session while limiting rebuffering and avoiding abrupt bitrate swings.

Congestion control. A congestion-control sender adjusts its transmission behavior from transport feedback observed on the path, such as delay, loss, acknowledgments, and delivery rate. An RL-based policy in this setting makes a new control decision at the beginning of each interval, whose duration is tied to the path dynamics. Sage [59] is the RL-based congestion-control system studied in this paper. Its reward combines throughput, latency, and packet loss, so the controller must push traffic efficiently without driving the path into persistent queueing or loss. Router-side mechanisms address a complementary control surface: Titrate adapts queue thresholds to balance throughput, latency, and burst absorption [62].

Load balancing. A load balancer in a replicated distributed store assigns each arriving request to one of several candidate servers. The balancer observes request-arrival patterns, recent request sizes, and its own estimate of outstanding work already sent to each server, but it does not directly observe each server's instantaneous internal utilization. Park [39] is the RL-based policy we study in this setting. Its goal is to route requests so that system-wide service remains efficient despite skewed arrivals, heterogeneous service speeds, and incomplete visibility into the servers' real-time state.

C Additional Implementation Details

C.1 Bilevel Search Implementations for Pensieve and Park

Adaptive bitrate streaming. Pensieve [38] instantiates the same formulation in a chunk-level ABR simulator. Here the unique design choice is not process synchronization, but an exact short-horizon reference oracle. At each chunk, u_t is the bandwidth used to download the next video chunk, and the simulator updates download time, buffer occupancy, rebuffering, sleep behavior, and QoE through Pensieve's standard dynamics. The solver controls only this bandwidth process. Pensieve still chooses bitrates through its normal observation interface, so the generated trace affects the controller only through measured throughput, delay, buffer state, next chunk sizes, and remaining chunks.

REGUARD applies Eqn. (10) exactly over a rolling window of K chunks. For the realized bandwidths and the buffer state at the beginning of the window, REGUARD enumerates candidate bitrate sequences in Π and selects the sequence with the largest QoE reward from Table 1. With six bitrate levels, this local search has size 6^K, which is tractable for small K and avoids the infeasible full-session search over 6^{48} sequences. This rolling oracle gives the outer solver dense feedback about avoidable ABR mistakes, such as over-aggressive bitrate increases before a bandwidth drop or conservative decisions that miss short high-bandwidth opportunities.

Load balancing. Park [39] instantiates the formulation as closed-loop workload generation for load balancing. At each load-balancing decision, u_t specifies the next inter-arrival time and job size, and the simulator advances queues, running jobs, completion events, and active-job-time reward after Park dispatches the current job. The action variables are typically log-scaled because both arrival gaps and job sizes span orders of magnitude. This lets the solver express bursts, idle periods, small jobs, and large jobs without leaving the controlled workload family.

Park uses a set of traditional heuristics as its portfolio for Π: Least Completion Time, Join Shortest Queue, and Choose Fastest Server. REGUARD applies Eqn. (10) by rolling each heuristic forward from the same simulator snapshot over the same generated workload window and selecting the best active-job-time reward. The distinctive implementation issue is snapshot-consistent counterfactual evaluation. The arrival stream, server speeds, current incoming job, and initial simulator state must be identical across policies, while each policy must have independent queues, running jobs, event timelines, and reward accounting. Deep snapshots enforce this separation, so a positive gap means that Park is dominated by simple decision rules under the same workload rather than by an artifact of shared mutable state. The resulting scenarios emphasize decision-sensitive workload patterns, such as bursts that make queue-length estimates misleading or size mixtures

where a poor server choice creates head-of-line blocking.

C.2 Protection Implementations for Pensieve and Park

Adaptive bitrate streaming. For Pensieve [38], the protected controller consumes the 6×8 ABR state summarized in Table 3. This state contains histories of bitrate, buffer, throughput, delay, and chunks remaining, plus the next-chunk size vector for the available bitrate choices. Pensieve's action space is discrete because each action selects one bitrate level. For this reason, REGUARD implements the generic adjustment as a learned safe action interval rather than as a continuous nudge. The rules infer lower and upper bitrate bounds $L(s_t)$ and $U(s_t)$, which define $\mathcal{A}_{\text{safe}}(s_t) := \{a \in \mathcal{A} : L(s_t) \leq \text{bitrate}(a) \leq U(s_t)\}$. If Pensieve's chosen bitrate lies inside this set, REGUARD abstains. If Pensieve chooses a bitrate above $U(s_t)$, REGUARD backs off by selecting Pensieve's highest-scored action inside $\mathcal{A}_{\text{safe}}(s_t)$. If Pensieve chooses a bitrate below $L(s_t)$, REGUARD pushes harder by selecting Pensieve's highest-scored action inside the safe set. Thus, REGUARD masks unsafe bitrate choices while preserving Pensieve's ranking among the allowed actions:

$$a_t^{\text{prot}} := \begin{cases} a_t, & a_t \in \mathcal{A}_{\text{safe}}(s_t), \\ \arg\max_{a \in \mathcal{A}_{\text{safe}}(s_t)} p_t(a), & a_t \notin \mathcal{A}_{\text{safe}}(s_t), \end{cases}$$

where $p_t(a)$ is Pensieve's action score or probability for bitrate action a.

Load balancing. For Park [39], the protected controller observes the load-proxy state summarized in Table 4 before dispatching an incoming job. The original policy input contains only per-server load proxies and the incoming job size. For rule learning, REGUARD derives summaries such as total load, mean load, load imbalance, server load ranks, and each server's load after hypothetically adding the incoming job from the same observation. These derived features are not additional inputs to the original Park policy. Park's action space is discrete because each action selects a server. REGUARD implements protection as a risk rule followed by a per-server allow mask. The learned rules define a safe action set $\mathcal{A}_{\text{safe}}(s_t) := \{i \in \mathcal{A} : \text{AllowServer}_i(s_t) = 1\}$. If no risk rule matches, all servers are treated as safe and REGUARD abstains. If Park's selected server is in $\mathcal{A}_{\text{safe}}(s_t)$, the action is left unchanged. If the state is risky and Park's selected server is outside the safe set, REGUARD selects the highest-scored Park action among the allowed servers:

$$a_t^{\text{prot}} := \begin{cases} a_t, & \begin{array}{c}\text{Risky}(s_t)=0 \\ \text{or } a_t \in \mathcal{A}_{\text{safe}}(s_t),\end{array} \\ \arg\max_{i \in \mathcal{A}_{\text{safe}}(s_t)} p_t(i), & \begin{array}{c}\text{Risky}(s_t)=1 \\ \text{and } a_t \notin \mathcal{A}_{\text{safe}}(s_t).\end{array} \end{cases}$$

This mapping realizes BACK_OFF and PUSH_HARDER as application-specific server redirections rather than scalar action changes. For example, a rule may prevent Park from sending another small job to an overloaded slow server and redirect the decision to the best Park-scored server among those supported by reference heuristics. REGUARD therefore constrains only risky dispatches while preserving Park's preferences whenever they are compatible with the learned safe set.

D Controller State Features

This appendix summarizes the controller state features referenced by the protection implementation in Section 5.

E Additional Evaluation Statistics

This appendix records the detailed numeric values, secondary comparisons, and per-method breakdowns omitted from Section 6.

E.1 Detailed Results for E1

For Pensieve, REGUARD reaches mean log performance gap 4.84, versus 4.00 for Random, 1.29 for Indago, 1.21 for Genet, 1.19 for Gilad et al., and 1.09 on Normal. On the original scale, that is 6.95× the Random gap and 5641× the Normal gap, with maximum 5.14. For Sage, REGUARD reaches mean relative performance gap 63.84%, versus 40.66% for Random and 1.66% on Normal, with maximum 70.21%. For Park, REGUARD reaches mean relative performance gap 43.49%, versus 7.05% for Gilad et al. and 0.29% on Normal, with maximum 203.80%.

E.2 Detailed Results for E2 and E3

On the most challenging family for each application, REGUARD removes 85.04% of the normalized performance gap for Pensieve, 80.01% for Sage, and 79.12% for Park. For transfer, the same REGUARD configuration removes 75.15% on Random and 22.09% on Indago for Pensieve, 90.08% on Gilad et al. and 98.54% on Genet for Sage, and 68.20% on Gilad et al. and 36.01% on Indago for Park. On Normal, REGUARD still reduces the normalized performance gap by 18.72% for Pensieve, 33.38% for Sage, and 36.11% for Park. By contrast, retraining baselines increase the Normal gap by up to 24.25% for Pensieve, approximately 15× for Sage, and 81.11% for Park. Fig. 6 provides the complementary ablation result. Across all three applications, the higher-gap counterfactual sources in that figure consistently yield higher average protection percentages, while weaker sources yield much weaker protection.

E.3 Detailed Results for E4 and E5

For Pensieve, mean decision-time ratio is 3.67% without protection and 11.22%, 32.36%, 17.88%, and 27.98% across Re-

Obs. index	Raw col.	Feature summary
0	2	Current smoothed RTT, normalized as RTT divided by 100 ms.
1	3	Current RTT variation in ms.
2	7	Current delivery rate, normalized by bandwidth factor.
3	9	TCP congestion-avoidance state.
4–12	10–18	RTT rolling summaries over short, medium, and long windows: average, minimum, and maximum.
13–21	19–27	Delivery-rate rolling summaries over short, medium, and long windows: average, minimum, and maximum.
22–30	28–36	Min-RTT-ratio and RTT-rate rolling summaries over short, medium, and long windows: average, minimum, and maximum.
31–39	37–45	RTT-variation rolling summaries over short, medium, and long windows: average, minimum, and maximum.
40–48	46–54	Inflight and unacked-data rolling summaries over short, medium, and long windows: average, minimum, and maximum.
49–57	55–63	Lost-packet rolling summaries over short, medium, and long windows: average, minimum, and maximum.
58	65	Time delta since the previous observation.
59	66	Current min-RTT-ratio or RTT-rate signal.
60	67	Current loss signal, normalized by bandwidth factor.
61	68	ACKed rate.
62	69	Delivery-rate growth ratio relative to the previous window.
63	70	Max-delivery-rate or cwnd-style utilization ratio.
64	71	Current windowed delivery rate, normalized by bandwidth factor.
65	72	cwnd-unacked ratio.
66	73	Max delivery-rate growth ratio relative to the previous max.
67	74	Max recent windowed delivery rate, normalized by bandwidth factor.
68	76	Previous Sage action or action feature.

Table 2: Sage policy observation features used by REGUARD. Sage consumes 69 selected entries from a larger 77-field shared-memory telemetry message. The raw fields excluded from the policy observation are timestamp, path or bandwidth scalar, RTO, ATO, pacing rate, slow-start threshold, one net-goodput field, and reward. In particular, reward is present in the raw shared-memory message but excluded from the policy observation.

Guard iterations 0–3. For Sage, the corresponding values are 41.37% without protection and 50.11%, 47.55%, 49.11%, and 48.28% across ReGuard iterations 0–3. For Park, the corresponding values are 2.00% without protection and 2.41%, 4.56%, 6.52%, and 1.85% across ReGuard iterations 0–3. Across all three systems, every iteration remains below 100% of the available decision budget, and later iterations do not induce systematic runtime growth despite stronger protection. For Pensieve, the mean gap drops from 4.84 at iteration 0 to 1.93, 1.22, and 1.24 across iterations 1–3, closing 96.6% of the distance to Normal by iteration 2. For Sage, the mean gap drops from 63.84% to 15.28%, 3.82%, and 3.78%, a 94.08% reduction by iteration 3 and a 96.59% reduction in distance to Normal. For Park, the mean gap drops from 33.45% to 6.16%, 3.21%, and 2.43%, a 92.73% reduction by iteration 3 and a 93.53% reduction in distance to Normal.

F Additional Failure Analyses

Due to space constraints, the main paper keeps only the Park case study in Section 7. This appendix contains detailed supporting statistics for Park and the corresponding Pensieve and Sage analyses. Below, pk denotes the corresponding kth-percentile predicate computed from normal scenarios (§5).

F.1 Park: Detailed supporting statistics

This subsection records the detailed quantitative support omitted from Section 7. In Fig. 9, the 107-decision slice consists of 69.2% jobs of at most 0.2KB, including 45.8% in the 0.126–0.525KB range that the fast servers can easily absorb, while only 10.3% are 5.0KB jobs appearing in four short bursts. For jobs of at most 0.2KB, Park chooses a slow server 97.3% of the time and server 1 alone 86.5% of the time, whereas Least Completion Time routes 81.1% of those same jobs to the fast tier and 71.6% to server 9. The large-gap region covers 76.6% of the slice. During that region, at least one fast server is idle on every step and server 9 is idle on every step, yet Park still sends traffic to a slow server on 86.0% of steps and to server 1 on 76.6% of steps. Inside that same region, Park never chooses a fast server, achieves mean chosen service rate 0.25 versus 0.97 for Least Completion Time, selects a zero-load server 0.0% of the time, and incurs mean regret 6.24.

Fig. 10 shows the same mechanism at workload scale. Under the challenging scenarios discovered by REGUARD, Park places 76.3% of all decisions on server 1, up from 5.6% on normal workloads. Its slow-server share rises from 29.1% to 81.5%, while its fast-server share falls from 53.1% to 14.1%. Least Completion Time under the same scenarios still routes 80.6% of decisions to the fast tier and 56.8% to server 9. At least one fast server remains idle on 98.0% of steps, yet Park still chooses a slow server with a fast server idle on 81.3% of

State row	Feature	Meaning	Normalization
0	Last selected bitrate	Bitrate selected for the most recently downloaded chunk.	$s_0 = b_t/\max_b$
1	Buffer occupancy	Current playback buffer size.	$s_1 = B_t/10$
2	Measured throughput	Throughput observed from the last chunk download.	$s_2 = S_t/(d_t \cdot 1000)$
3	Download delay	Time taken to download the last chunk.	$s_3 = d_t/(1000 \cdot 10)$
4	Next chunk sizes	Byte sizes of the next video chunk at each available bitrate level.	$s_4[a] = S_{t+1}(a)/1000^2$
5	Chunks remaining	Number of chunks remaining in the video.	$s_5 = \min(C_t, 48)/48$

Table 3: Pensieve policy observation features used by REGUARD. Pensieve represents state as a 6×8 matrix: six feature channels over the most recent eight decision steps. The bitrate levels are 300, 750, 1200, 1850, 2850, and 4300 Kbps. Here B_t is buffer seconds, S_t is downloaded chunk size in bytes, d_t is download delay in ms, $S_{t+1}(a)$ is the next-chunk size at bitrate action a, and C_t is chunks remaining. The observation contains history for bitrate, buffer, throughput, delay, and chunks remaining, plus the current next-chunk size vector for the six bitrate choices.

Component	Feature form	Meaning
Policy observation	$s_t = [L_0, L_1, \ldots, L_{N-1}, J_t]$	Park observes one load proxy per server and the incoming job size, with $N = 10$ in the default setup.
Server load proxies	`LoadProxy0`, …, `LoadProxy9`	Each L_i estimates the observable load on server i.
Load-proxy value	$L_i = \sum_{j \in Q_i} \text{size}(j) + \mathbf{1}[\text{server } i \text{ is running}] \cdot \max(0, \text{finish_time}_i - t)$	The proxy combines queued job sizes with the remaining time of the currently running job.
Incoming job	`IncomingSize`, with $J_t = \text{size}(\text{incoming job})$	The size of the job currently waiting to be assigned.
Clipping	`obs_high`, typically 500000.0	All observation values are clipped by the observation upper bound.
Not in policy input	Future arrivals, oracle gap, reference actions, explicit queue lengths, server identities beyond vector position, and rule labels	These quantities are not part of the original Park policy observation.
Rule-derived features	Total load, mean load, minimum and maximum load, load ranks, and load-plus-incoming per server	These interpretable features are derived from the same observation for rule learning and are not extra inputs to the original Park policy.

Table 4: Park policy observation and rule-derived features. The original Park policy sees only per-server load proxies plus the incoming job size. REGUARD can derive additional interpretable predicates from the same observation without changing the policy input.

steps and chooses server 1 while server 9 is idle on 60.8% of steps.

The recovery rules also show that reopening the fast tier is not limited to a single server. One additional correction rule is

$$(\text{Load}_9 \le \text{p25}) \wedge (\text{PostLoad}_2 \le \text{p25}) \\ \wedge (\text{MeanLoad} \le \text{p25}) \wedge (\text{StdLoad} \le \text{p25}) \implies \text{Allow}_7 = \texttt{true}.$$

This rule says that one fast server already looks light, another candidate would still remain light after taking the job, and the overall cluster is neither heavily loaded nor badly imbalanced. It shows that REGUARD is reopening the fast tier rather than hard-coding a single dispatch.

F.2 Pensieve: Sparse-bandwidth regimes trigger bitrate overshoot

Pensieve's failure is sustained over-aggression under long sparse-bandwidth regimes rather than a few isolated bitrate mistakes. Fig. 11 shows scarce-bandwidth spans that cover 35.5% of a trace duration but contain 78.6% of all 6,248 bitrate decisions. This concentration matters because the failure occupies a dominant operating regime instead of a few outliers. Within that regime, Pensieve selects 465.6Kbps on average, whereas the oracle selects 304.2Kbps, so Pensieve is 53.1% higher. When Pensieve overshoots, the overshoot is substantial rather than marginal. On the 21.3% of decisions where Pensieve exceeds the oracle, it averages 1073.0Kbps versus the oracle's 308.8Kbps, with a mean gap of 764.2Kbps and a median gap of 900Kbps. Even inside scarce-bandwidth periods alone, Pensieve still averages 422.6Kbps versus 300.9Kbps for the oracle and chooses at least 1200Kbps on 10.4% of steps, while the oracle does so on only 0.04%. The largest mismatch reaches 4300Kbps for Pensieve versus 300Kbps for the oracle under realized bandwidth of only 0.001Mbps.

The mechanism behind this failure is straightforward. Under repeated bandwidth scarcity, Pensieve keeps treating the state as compatible with aggressive upgrades even after multiple recent throughput samples collapse and chunk-download delays explode. The result is not merely a suboptimal bitrate ranking. It is a repeated decision to request chunks whose sizes the sparse link cannot support, which magnifies rebuffering and drives the large performance gap in Fig. 11.

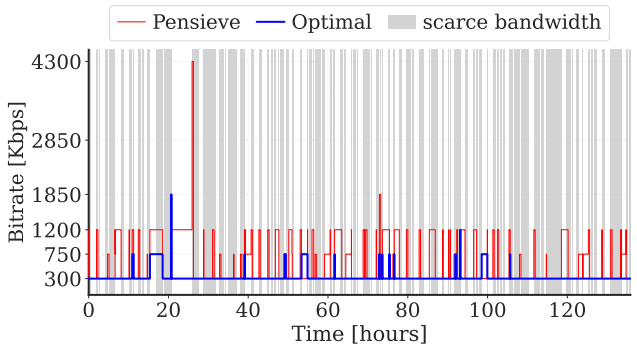


Figure 11: Sparse-bandwidth scenarios expose a systematic over-aggressive bitrate policy in Pensieve. Even when the realized link remains near the bottom of the bitrate ladder, Pensieve repeatedly selects much larger rates than the oracle, creating a large avoidable performance gap.

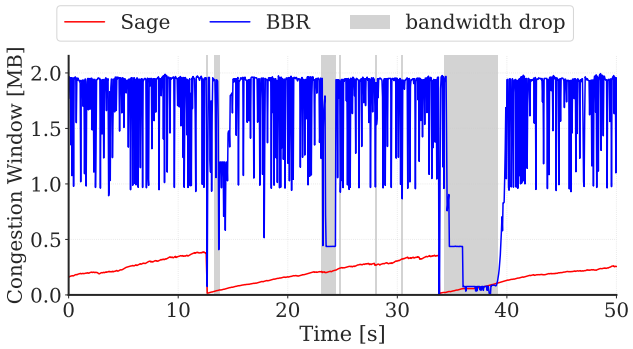


Figure 12: Sharp bandwidth drops expose Sage's slow recovery. Sage collapses its congestion window far more than BBR and reopens it much more slowly, leaving throughput far below what the path can still support.

The logic rules used by REGUARD make that mechanism explicit. One representative rule is

$$(\text{Buffer} \le \text{p10}) \wedge (\text{Throughput}_4 \le \text{p10}) \\ \wedge (\text{Delay}_3 > \text{p95}) \wedge (\text{NextChunk}_3 > \text{p90}) \implies \text{Bitrate} \le 750.$$

This rule says that when the buffer is already shallow, several recent throughput samples are in the bottom decile, recent download delays are in the worst 5%, and even the next mid-range chunk is large, Pensieve should not climb above 750Kbps. Its semantic meaning matches the failure directly: sparse bandwidth is not a one-step dip, so aggressive upgrades only deepen the mismatch between requested chunks and available capacity.

A second rule is an even harder clamp:

$$(\text{Throughput}_5 \le \text{p25}) \wedge (\text{Throughput}_6 \le \text{p10}) \\ \wedge (\text{Delay}_7 > \text{p95}) \wedge (\text{NextChunk}_2 > \text{p10}) \implies \text{Bitrate} \le 300.$$

This rule says that when several consecutive throughput samples are near the floor, the most recent delay is extreme, and even moderate bitrate choices imply large chunks, the next action should be capped at the minimum bitrate. That is precisely the kind of human-readable explanation the figure calls for. The rule does not say that the network is simply bad. It says that this particular combination of sparse recent throughput, severe delay, and large pending chunks is a high-impact state in which Pensieve's aggressive policy is likely to fail.

F.3 Sage: Sharp drops trigger overreaction and slow reopening

Sage's failure is not generic weakness on a challenging path. As Fig. 12 shows, the dominant mechanism is an excessive reaction to bandwidth drops followed by very slow reopening. Across the plotted 50s span, Sage's congestion window averages 0.20MB, or 139 packets, whereas BBR averages 1.56MB, or 1066 packets. The medians are 0.21MB for Sage and 1.94MB for BBR, so BBR's window is 9.4× larger at the median and larger than Sage's at 96.9% of sampled times. The sharpest drop, from 150Mbps to 5Mbps at 12.6s, makes Sage collapse from 261 packets to 11 packets. Sage then reaches only 50 packets after 2.45s, 100 packets after 6.3s, and half of its pre-drop window only after 9.15s. BBR also cuts back at that instant, but it rebounds to 827 packets within 50ms and to 1342 packets within 100ms, regaining half of its pre-drop window in just 50ms. Over the first second after the drop, Sage averages only 19 packets while BBR averages 1097 packets, a 58× gap. Even after the longest 5Mbps episode ends, BBR climbs back above 1000 packets within 0.95s of bandwidth returning to 150Mbps, whereas Sage still needs 1.85s merely to exceed 100 packets.

The key weakness is therefore not the initial backoff alone. It is that Sage continues acting as if the path remains dangerous long after the immediate drop has passed. That persistent conservatism keeps its congestion window tiny, which in turn keeps realized throughput tiny. Fig. 12 therefore exposes a specific control failure: Sage brakes hard on sudden drops and then reopens far too slowly.

The onset of that failure appears in rules such as

$$(\text{LossDelta} \ge \text{p95}) \wedge (\text{LossMin} \ge \text{p95}) \\ \wedge (\text{Rtt} \le \text{p25}) \wedge (\text{WindowedRate} \le \text{p10}) \implies \text{PUSH_HARDER}.$$

This rule says that loss has just surged, but RTT is still low while the windowed delivery rate has already fallen into the bottom decile. That combination is a compact signature of overreaction. The controller has already slammed on the brakes, yet the latency signal does not justify staying that conservative.

The aftermath appears in rules such as

$$(\text{LossMax} \le \text{p25}) \wedge (\text{LossMin} \le \text{p25}) \\ \wedge (\text{MaxRate} \le \text{p10}) \wedge (\text{AvgRate} \le \text{p10}) \implies \text{PUSH_HARDER}.$$

This rule is important because it no longer describes an acute crisis. Loss is now low, RTT inflation is not severe, and yet both the recent maximum and the recent average delivery rates remain near the floor. In plain language, the network is no longer screaming "back off," but Sage is still behaving as though it is.

A third rule isolates the slow-recovery phase even more directly:

$$(\text{CurrentRate} \leq \text{p10}) \wedge (\text{RateGrowth} \leq \text{p10}) \\ \wedge (\text{AvgRate} \leq \text{p10}) \wedge (\text{RateVsMax} \leq \text{p25}) \implies \text{PUSH_HARDER}.$$

This rule says that Sage is not only small in absolute terms. It is growing slowly and operating far below what it was recently capable of. That is exactly the behavior seen in Fig. 12, and it makes the failure interpretable in a way that the raw congestion-window trace alone cannot.

G Testbed

For fair evaluation, we conduct all experiments on a two-socket server with 40 logical Intel Xeon E5-2660v3 CPUs running at 2.60GHz and 256GiB of DRAM.